

Universidade de Vigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D. Nome Alumna/o

para a obtención do Título de Graduado en Enxeñaría Informática

Título do Traballo de Fin de Grado



Xullo, 20XX

Traballo de Fin de Grao N°:

Titor/a:

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática

Agradecimientos

[Texto]

Dedicatoria

[Texto]

Índice

1. Introducción	6
2. Objetivos	7
2.1. Objetivo principal	7
2.2. Objetivos específicos	7
2.3. Alcance y limitaciones	7
3. Antecedentes y contexto	8
3.1. Deep reinforcement learning en ajedrez	8
3.2. Transformers en ajedrez	8
4. Resumen de la solución propuesta	9
5. Marco teórico y práctico	10
5.1. Marco teórico	10
5.1.1. Diseño de hipótesis	10
5.1.2. Entorno: reglas del ajedrez 5x5	10
5.1.3. Arquitectura basada en transformers: encoder y self-attention	13
5.1.4. Modelado del aprendizaje por refuerzo	13
5.2. Marco práctico	15
5.2.1. Codificación del espacio de estados (input)	15
5.2.2. Codificación del espacio de acciones (output)	15
5.2.3. Arquitectura transformer	17
5.2.4. Algoritmo de RL: Proximal Policy Optimization (PPO)	18
5.2.5. Descripción del conjunto de datos	21
5.2.6. Herramientas y tecnologías empleadas	22
5.3. Diseño experimental	23
5.3.1. Diseño experimental para hipótesis 2	23
5.3.2. Diseño experimental para la hipótesis 3	24
6. Planificación y seguimiento	26
7. Principales aportaciones	27
7.1. Efectos de la representación sobre la eficiencia y calidad del aprendizaje	27
7.1.1. Evaluación sobre el conjunto de test	27
7.1.2. Test de significatividad estadística	28
7.1.3. Evolución y curvas de aprendizaje	29
7.1.4. Evaluación en juego real	30
7.1.5. Discusión de los resultados	31
7.1.6. Conclusión sobre la hipótesis 2	33
7.2. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo	34
7.2.1. Resultados del entrenamiento	35
7.2.2. Evolución del Elo y razones de finalización de partida	37
7.2.3. Evaluación estadística de la fuerza de juego final	39
7.2.4. Resultados del torneo mixto de 8 agentes	39
7.2.5. Conclusión sobre la hipótesis 3	42

7.3. Resultados generales en el entorno de Minichess	42
8. Conclusiones	42
9. Vías de trabajo futuro	44
Referencias	47
A. Anexos	49
A.1. Hiperparámetros del entrenamiento y estudio de ablación	49
A.2. Estadísticas y análisis del conjunto de datos	51
A.3. Resultados adicionales del estudio de ablación sobre un modelo mayor	53
A.3.1. Hiperparámetros de la red de mayor capacidad	54
A.3.2. Evaluación en test holdout	54
A.3.3. Análisis estadístico y test de Tukey HSD	54
A.3.4. Curvas de aprendizaje del modelo grande	56
A.3.5. Torneo del modelo grande	56
A.4. Formulación matemática de las funciones de pérdida	58
A.5. Evolución de las pérdidas por cabeza durante el entrenamiento	58
A.6. Resultados detallados de los tests de significatividad estadística	59
A.6.1. Test de normalidad (Shapiro-Wilk)	59
A.6.2. Test de diferencias globales (One-Way ANOVA)	59
A.6.3. Comparaciones múltiples por parejas (Tukey HSD)	60
A.7. Hiperparámetros PPO	61
A.8. Instrucciones de ejecución y reproducibilidad	62

Índice de figuras

1.	Posición inicial del ajedrez Gardner.	11
2.	Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c... . . .	15
3.	Visualización de codificaciones de salida. De izquierda a derecha: a, b, c... . . .	16
4.	Diagrama de la arquitectura Transformer propuesta	19
5.	Ejemplo del conjunto de datos	22
6.	Distribución de las métricas de test	29
7.	Curvas de aprendizaje promediadas	30
8.	Matrices de winrate del torneo	31
9.	Curvas de aprendizaje de PPO	36
10.	Evolución conjunta del Elo relativo	37
11.	Desenlaces de partida frente al baseline heurístico	37
12.	Desenlaces de partida frente al baseline aleatorio	38
13.	Distribución de Elo final	39
14.	Matrices de tasa de victoria del torneo mixto	40
15.	Desenlaces de partida del torneo mixto	41
16.	Distribución de resultados globales y tipo de movimientos.	51
17.	Distribución de balances materiales y densidad de piezas.	52
18.	Distribución espacial de las posiciones de los reyes en el tablero.	52
19.	Distribución de movimientos: casilla de origen	52
20.	Distribución de movimientos: casilla de destino	53
21.	Distribución y variabilidad de las métricas en test para el modelo grande	56
22.	Curvas de aprendizaje promediadas para el modelo grande	56
23.	Matrices cruzadas de tasas de victorias para el torneo del modelo grande	57
24.	Evolución de las pérdidas de entrenamiento desglosadas por cabeza de salida en el estudio de ablación	58

Índice de cadros

1.	Rendimiento medio y desviación estándar sobre el conjunto de test	28
2.	Resultados del torneo de 100 partidas	31
3.	Comparación estadística de convergencia	39
4.	Resultados del torneo mixto de 8 agentes	40
5.	Hiperparámetros fijos del estudio de ablación	50
6.	Parámetros y complejidad de modelos	53
7.	Hiperparámetros del modelo grande	54
8.	Métricas de test para el modelo grande	55
9.	Elo y clasificación para el modelo grande	57
10.	Comparación estadística usando Tukey HSD para la política	60
11.	Comparación estadística usando Tukey HSD para el valor	60
12.	Hiperparámetros del algoritmo PPO	61

1. Introducción

“Chess is a game of luck.
If you have a good opponent you have
bad luck, and if you have a bad
opponent you have good luck.”^[1]

Jacob Segal, 7 años

Este Trabajo de Fin de Grado consiste en la aplicación y estudio del Aprendizaje por Refuerzo (RL) sobre una variante de ajedrez simplificada, el ajedrez 5×5 (también llamado ajedrez de Gardner o Minichess). El objetivo central es validar el uso de arquitecturas Transformer en tareas de RL, estudiando el efecto de las representaciones del tablero y comprobando si el preentrenamiento supervisado mejora la convergencia del agente, y mostrando como, aún siendo un problema reducido, esta versión del ajedrez presenta desafíos y complejidades suficientes para la investigación de técnicas de RL.

Dado que el ajedrez 8×8 requiere una capacidad computacional y un tiempo de ejecución que excede con creces los recursos disponibles en el marco del trabajo, se ha optado por utilizar el ajedrez 5×5 . Este entorno reducido mantiene las dinámicas esenciales del aprendizaje por refuerzo adversarial, como las acciones legales, las recompensas diferidas y la planificación estratégica, pero se mantiene computacionalmente manejable para la experimentación ágil con hardware de consumo.

A nivel personal este trabajo busca, además de lo anteriormente mencionado, proporcionar una oportunidad para entender mejor el proceso de investigación y desarrollo de un proyecto de IA, desde la formulación de las hipótesis hasta la evaluación de los resultados. También busca servir para aprender acerca del diseño y programación de redes neuronales avanzadas con el uso de mecanismos de atención, y servir como un “simulacro” de los procesos de preentrenamiento seguidos de aprendizaje por refuerzo, los cuales son muy comunes a día de hoy en aplicaciones como los modelos de lenguaje, modelos multimodales, o ciertos sistemas de recomendación, e incluso procesos como el *imitation learning* en juegos de acción continua o entornos de robótica.

El resto de este documento se estructura de la siguiente manera: en la Sección 2 se detallan los objetivos generales y específicos junto con el alcance del trabajo; en la Sección 3 se describe el estado del arte en transformers, aprendizaje por refuerzo y ajedrez y los antecedentes del problema; en la Sección 4 se introduce y resume la solución metodológica planteada.

En la Sección 5 se desarrolla el marco teórico (5.1) y práctico (5.2), detallando las hipótesis a probar, el modelado matemático del entorno de ajedrez, de la arquitectura transformer, la codificación de entradas y salidas y del problema de RL, además de justificar las herramientas utilizadas para el desarrollo. En la Sección 6 se muestra la planificación temporal del proyecto.

Las secciones finales presentan las aportaciones principales y resultados empíricos de los experimentos (Sección 7), las conclusiones y discusión entorno a los resultados (Sección 8) y las futuras líneas de investigación y limitaciones del trabajo (Sección 9). Los anexos contienen información adicional específica, o desarrollos / cuestiones que van mas allá del alcance del trabajo (Sección A).

2. Objetivos

2.1. Objetivo principal

Diseñar, entrenar y evaluar un agente de Inteligencia Artificial basado en arquitecturas Transformer mediante Aprendizaje por Refuerzo para el entorno de ajedrez 5×5 . Probar la viabilidad del entorno de ajedrez 5×5 en un contexto de investigación entorno a RL, demostrar la mejora proporcionada por sesgos inductivos en modelos de ajedrez, y confirmar la ventaja de modelos preentrenados frente a modelos entrenados desde cero en PPO.

2.2. Objetivos específicos

1. Implementar un algoritmo de RL para ajedrez 5×5 , junto a la lógica necesaria para self-play.
 2. Desarrollar y adaptar una arquitectura Transformer para procesar estados del tablero y predecir distribuciones de probabilidad sobre el espacio de acciones y valores de estado.
 3. Obtener datos adecuados para el preentrenamiento de un modelo de ajedrez Gardner.
 4. Preentrenar de forma eficiente un modelo transformer mediante aprendizaje supervisado.
 5. Evaluar empíricamente el impacto de distintas representaciones del tablero y del espacio de acciones en la velocidad de aprendizaje y rendimiento del modelo.
 6. Comparar el rendimiento y la eficiencia de convergencia entre un agente entrenado mediante RL desde una inicialización aleatoria (*tabula rasa*) frente a un agente con inicialización mediante aprendizaje supervisado.
-

2.3. Alcance y limitaciones

El alcance del trabajo se ha enmarcado en los siguientes límites prácticos e investigativos:

- **Entorno de juego:** Se limita estrictamente a la variante Gardner de ajedrez 5×5 . No se explorarán otras variantes del miniajedrez ni el ajedrez estándar 8×8 .
- **Recursos computacionales:** El entrenamiento de los modelos se realizará utilizando hardware de consumo accesible (GPUs de gama media) con un límite temporal de entrenamiento acotado a un número de horas que permita la experimentación iterativa ágil.
- **Algoritmos de aprendizaje:** Se prioriza el uso de métodos de optimización basados en gradiente de política (PPO), evaluando el *Model-Free RL*. No se hará uso de algoritmos basados en planificación o búsqueda explícita como MCTS (Monte Carlo Tree Search), con el fin de optimizar el tiempo de ejecución en hardware de consumo.

3. Antecedentes y contexto

El Aprendizaje por Refuerzo (RL) ha demostrado en los últimos años resultados extraordinarios en numerosos campos como el Go, el ajedrez, la síntesis de proteínas, los modelos de lenguaje y la robótica. Con la enorme capacidad de computación actual, que sigue creciendo día a día, el RL aprovecha las características enunciadas en *The Bitter Lesson* de Sutton^[2]: la búsqueda masiva y el aprendizaje de funciones a través del muestreo superan a largo plazo a los enfoques basados en conocimiento humano inyectado manualmente.

Ambos procesos, búsqueda y aprendizaje, son altamente exigentes en términos de computación, pero bajo la suposición de que esta aumenta al ritmo de la Ley de Moore, los métodos que aprovechan la computación superan eventualmente aquellos basados en conocimiento humano. Esto es especialmente cierto en entornos (discretos o continuos) con espacios de estado masivos, como son el ajedrez o la robótica.

3.1. Deep reinforcement learning en ajedrez

Desde la irrupción del aprendizaje profundo y el aprendizaje por refuerzo en los juegos de tablero, se han explorado diversas arquitecturas neuronales. El hito inicial lo marcó AlphaGo^[3], que combinó redes convolucionales (CNNs) con búsqueda de árbol Monte Carlo (MCTS) para vencer al campeón mundial de Go. Posteriormente, esta metodología se generalizó con AlphaZero^[4], un algoritmo de RL capaz de dominar el ajedrez, el shogi y el Go partiendo desde cero y utilizando una única red neuronal para evaluar posiciones y priorizar jugadas. El impacto de estos métodos fué más allá, al demostrar el poder del *deep reinforcement learning* en otros dominios científicos complejos, como la predicción de estructuras de proteínas mediante AlphaFold^[5].

Paralelamente, en el ámbito de los motores de ajedrez tradicionales, enfoques como NNUE (*Efficiently Updatable Neural Networks*)^[6] tomaron un camino alternativo: en lugar de grandes redes convolucionales, emplean arquitecturas *feed-forward* muy ligeras con sesgos inductivos específicos del ajedrez, optimizadas para ejecutarse en CPU y priorizar la velocidad de evaluación y la profundidad de búsqueda táctica.

En estos motores de juego, las redes neuronales actúan como la parte *estratégica* mientras que la búsqueda actúa como la parte *táctica*, decidiendo a corto plazo bajo una serie de restricciones más claras (jugadas forzadas, amenazas claras como jaques o pérdida de material), y depende más del *cálculo*. Esto es, en cierto modo, reminiscente de cómo los humanos estructuramos nuestro pensamiento a la hora de jugar al ajedrez.

3.2. Transformers en ajedrez

Entre los enfoques anteriores, ninguno hacía uso de mecanismos de atención ni de arquitecturas basadas en Transformers, lo que nos lleva a explorar el potencial de estos últimos. Más recientemente, tras el éxito de esta arquitectura en los modelos de lenguaje (LLMs), han surgido aproximaciones que buscan aplicarla a tareas de RL y ajedrez. En este trabajo, se busca demostrar que la arquitectura Transformer es altamente competitiva para la tarea de RL, utilizando un entorno 5×5 para facilitar la simulación y el muestreo iterativo.

En este ámbito, Ruoss et al.^[7] demostraron la viabilidad de realizar planificación amortizada utilizando Transformers a gran escala aplicados al ajedrez. Por otro lado, Monroe y Chalmers^[8] desarrollaron un modelo basado en Transformers para dominar el ajedrez, sentando las bases para Chessformer, una arquitectura unificada propuesta posteriormente por Monroe et al.^[9]. Además, Jenner et al.^[10] aportaron evidencia sobre la existencia de capacidades de anticipación y planificación aprendidas de forma implícita por este tipo de redes, mientras que el proyecto open source Leela Chess Zero también ha incorporado y evaluado estas arquitecturas en su motor de juego^[11].

Por su parte, el estudio del miniajedrez o ajedrez en tableros reducidos tiene una larga trayectoria. Existen miles de variantes del ajedrez; la variante de Gardner fue propuesta originalmente por Martin Gardner en 1969^[12]. Este tipo de entornos se ha empleado tradicionalmente para analizar la complejidad matemática del juego y estudiar algoritmos de búsqueda heurística, al reducir drásticamente el espacio de estados sin perder la naturaleza del ajedrez convencional; necesidad de táctica y estrategia, y emergencia de dinámicas interesantes al contar con distintos tipos de pieza (lo que aumenta el número de combinaciones posibles). El ajedrez 5×5 es particularmente interesante ya que permite mantener todos los tipos de pieza (torre, alfil, caballo, dama y rey) y una disposición inicial similar a la del ajedrez estándar.

En 2013, Mehdi y Prost resolvieron débilmente el ajedrez Gardner, esto es, obtuvieron un algoritmo que permite asegurar el resultado de la partida partiendo de la posición inicial, pero no desde cualquier posición arbitraria. Utilizando técnicas de búsqueda e intervención humana, obtuvieron un oráculo para el jugador blanco y negro que asegura un empate partiendo de la jugada inicial.^[13]

4. Resumen de la solución propuesta

El desarrollo del trabajo se estructura en torno a la validación de las tres hipótesis planteadas 5.1.1. En primer lugar, se mide el impacto de la representación en un estudio de ablación supervisado, comparando codificaciones planas contra espaciales en la entrada, y políticas simples contra factorizadas en la salida. Tercero, se compara la velocidad de convergencia y la fuerza de juego de un bucle de PPO al inicializar la red desde cero frente a usar un preentrenamiento supervisado. El estudio de estas dos hipótesis permite analizar si el miniajedrez 5x5 conserva dinámicas estratégicas suficientes para servir como entorno de experimentación de aprendizaje por refuerzo con recursos limitados. De esta forma, el modelado y el diseño de la arquitectura detallados en la Sección 5 conectan directamente con los torneos y análisis estadísticos de la Sección 7.

5. Marco teórico y práctico

5.1. Marco teórico

5.1.1. Diseño de hipótesis

Para dar respuesta a los objetivos planteados, el desarrollo práctico se articula en torno a la demostración de las siguientes hipótesis experimentales, las cuales estructuran el diseño experimental del trabajo:

- **Hipótesis 1 (Reducción del problema):** El ajedrez 5x5 como entorno simplificado de RL puede conservar suficientes dinámicas del aprendizaje por refuerzo adversarial, y justificar el uso de redes neuronales con mecanismos de atención, al tiempo que se mantiene computacionalmente manejable para la experimentación.
 - **Hipótesis 2 (Representación):** El uso de representaciones estructuradas del tablero y del espacio de acciones (ej. planos espaciales vs. predicción por casillas origen/destino) mejora la eficiencia del aprendizaje frente a representaciones planas.
 - **Hipótesis 3 (Inicialización):** Un agente inicializado mediante aprendizaje supervisado a partir de jugadas generadas por un motor de referencia alcanza un nivel de rendimiento competitivo en menos *epochs* que un agente entrenado de forma puramente *tabula rasa*.
-

5.1.2. Entorno: reglas del ajedrez 5x5

El ajedrez 5×5 en su variante de Gardner es una reducción del ajedrez estándar. Para alguien familiar con el ajedrez, la variante de Gardner es idéntica, excepto que se recorta el tablero a un tamaño inferior, y se mantiene la distribución de las piezas de la columna a hasta la e, eliminando el enroque y el avance doble de peón. En caso de no conocer las reglas del ajedrez, a continuación se detallan las de esta variante:

- **Tablero:** tamaño de 5×5 casillas (columnas *a* a *e*, filas 1 a 5).
- **Disposición inicial:** La misma que el ajedrez estándar, pero con las piezas del lado de rey eliminadas, y una sola fila de separación entre peones. Cada jugador cuenta con 10 piezas. En la fila 1 (blancas) y fila 5 (negras) se colocan las piezas mayores en el orden: Torre (*R*), Caballo (*N*), Alfil (*B*), Dama (*Q*) y Rey (*K*). Los peones correspondientes se colocan en la fila 2 (blancas) y fila 4 (negras). La disposición de tablero se puede representar mediante una notación que veremos a continuación.
- **Omisión de reglas especiales:** No existe el enroque ni el avance doble de peón. Como consecuencia directa, tampoco existe la captura al paso (*en passant*).
- **Coronación de peones:** Cuando un peón alcanza la fila límite opuesta (fila 5 para blancas, fila 1 para negras), es obligatorio coronarlo a una de las siguientes cuatro piezas: Dama, Torre, Alfil o Caballo.
- **Condiciones de fin de partida:** La partida finaliza por (A) jaque mate (el rey está amenazado y no puede retirarse a una casilla a salvo, cubrirse, o capturar a la pieza que lo amenaza), (B) rey ahogado (el rey no está en jaque pero el jugador no tiene ningún movimiento legal), (C) insuficiencia de material para dar jaque mate (posición

muerta), (D) si pasan 75 movimientos sin capturas ni avances de peón, o (E) tablas por quintuple repetición (cinco posiciones iguales a lo largo de toda la partida (no necesariamente consecutivas)).

- **Condiciones reclamables**: la regla de las 5 repeticiones y de los 75 movimientos tiene sus contrapartes “reclamables”: la regla de las 3 repeticiones y la de los 50 movimientos requieren que uno de los jugadores reclame tablas para activarse.

Esta variante mantiene la naturaleza estratégica y la profundidad del ajedrez, pero en una fracción de su complejidad de cálculo original.

Representación de estados mediante notación FEN

Para interactuar con el entorno de ajedrez y proporcionar datos a los modelos de aprendizaje, se utiliza la notación FEN (*Forsyth-Edwards Notation*). Esta notación describe una posición particular del tablero mediante una cadena de texto que codifica la ubicación de las piezas, el jugador al que le toca mover y las disponibilidades de reglas especiales. En el contexto del ajedrez 5x5, el FEN se simplifica ya que cierta información no es aplicable a la variante de Gardner (como los enroques o las capturas al paso).

El FEN inicial de la variante Gardner se representa como:

```
rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1
```

La Figura 1 ilustra la disposición física y la colocación de las piezas en el tablero Gardner al inicio de la partida.



Figura 1: Posición inicial del ajedrez Gardner.

El ajedrez como juego y proceso teórico de decisión

El ajedrez es un juego de información perfecta y completa. En todo momento los jugadores tienen información total acerca de las funciones de utilidad de su rival y del estado de la partida, tanto presente como pasado. Esto proporciona fundamentos teóricos sólidos para probar algoritmos de RL; un entorno determinista y con funciones de transición bien definidas.

Dado el factor de ramificación estimado del ajedrez estándar (media de 35 hijos por nodo) y la complejidad del árbol de juego (10^{120}), propuesta por Claude Shannon en su artículo fundacional^[14], la variante de ajedrez 5×5 se establece como un entorno matemáticamente equivalente en sus reglas de transición, pero con un espacio de estados y una complejidad del árbol de juego acotados.

Para estimar la complejidad de la variante Gardner 5×5 , siguiendo un razonamiento similar al de Shannon, se pueden calcular las siguientes métricas:

- **Espacio de estados $\mathcal{S}$:** Mehdi y Prost estimaron que el número total de posiciones legales en el ajedrez 5×5 es aproximadamente 9×10^{18} , descontando posiciones ilegales (reyes en jaque simultáneo, peones en las filas de inicio/coronación, etc...) [13].
- **Complejidad del árbol de juego:** Realizando un análisis estadístico simple sobre los datos de entrenamiento de este trabajo, (10 millones de posiciones y 1.177.627 partidas completas) se obtiene un factor de ramificación promedio de $b \approx 8,78$ jugadas legales por posición y una profundidad promedio de partida de $d \approx 87,82$ medios movimientos.

Siguiendo la estimación de Shannon b^d , la complejidad del árbol de juego para Gardner se sitúa en:

$$N_{\text{media}} \approx 8,78^{87,82} \approx 7,2 \times 10^{82}$$

Este valor es 38 órdenes de magnitud inferior al límite superior del ajedrez convencional (10^{120}); mantiene una complejidad numérica suficiente como para mantener prohibitiva la resolución del juego por fuerza bruta, al mismo tiempo que permite una experimentación y convergencia de aprendizaje más rápida que el ajedrez estándar en hardware de consumo.

Dada la complejidad del entorno, nos gustaría poder modelarlo de modo que el aprendizaje se realice únicamente sobre el estado actual, ya que comprobar el historial de jugadas completo tendría un coste computacional excesivo. Buscamos que la probabilidad de alcanzar un estado futuro dependa únicamente del estado presente, y no del historial de estados anteriores. Esta es la propiedad de Markov.

El ajedrez como proceso de decisión de Markov (MDP)

Un proceso estocástico cumple la propiedad de Markov si la probabilidad de alcanzar un estado futuro, s_{t+1} , depende únicamente del estado presente s_t , siendo condicionalmente independiente del historial de estados anteriores. Formalmente:

$$P(s_{t+1}|s_t, s_{t-1}, \dots, s_0) = P(s_{t+1}|s_t)$$

Si definimos el estado del ajedrez únicamente por la disposición física de las piezas en el tablero temos que $s_t = B_t$, entonces el juego no es estrictamente un MDP. Recordemos las reglas mencionadas anteriormente: la triple repetición y la regla de los 50 movimientos tienen en cuenta el historial de estados, no solo el actual.

Para modelar el ajedrez 5×5 como un MDP, es necesario expandir el espacio de estados. Definimos el nuevo estado expandido $s \in \mathcal{S}$ como una tupla $s = (B, p, h)$ donde:

1. B : La disposición exacta de todas las piezas en el tablero de 5×5 en el instante t .
2. p : Un contador, el *half-move clock* o reloj de media jugada, que registra el número de medios movimientos realizados desde la última captura o avance de peón.
3. h : Un registro de las posiciones previas ocurridas desde la última transición irreversible (o, de forma computacionalmente más eficiente, un mapa de frecuencias que contabilice cuántas veces se ha visitado la disposición B actual en la partida en curso).

Al encapsular explícitamente los contadores p y h dentro de la representación del estado actual s_t , el modelo no necesita consultar cronológicamente cómo se llegó a dicho estado. Toda la información temporal y causal necesaria para evaluar la legalidad de las jugadas futuras o detectar empates está autocontenida en el instante presente. Bajo esta formulación, se satisface la propiedad de Markov, lo que nos permite simplificar el modelado.

5.1.3. Arquitectura basada en transformers: encoder y self-attention

A diferencia de los modelos de lenguaje (LLMs) que generan secuencias de texto token a token utilizando una arquitectura de decodificador con enmascaramiento causal, la evaluación de un tablero de ajedrez requiere una comprensión bidireccional del estado completo en un instante de tiempo / posición. No es necesario enmascarar la atención para evitar observar tokens futuros, ya que por la propiedad de Markov demostrada anteriormente, una posición no depende de estados posteriores ni anteriores.

Por ello, la arquitectura propuesta se fundamenta en un **Transformer Encoder**. Este modelo procesa simultáneamente todas las casillas del tablero, permitiendo que cada pieza calcule su relevancia respecto a todas las demás sin restricciones de enmascaramiento. El núcleo de esta arquitectura es el mecanismo de *Self-Attention*^[15], definido formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Donde Q (Query), K (Key) y V (Value) son matrices obtenidas mediante proyecciones lineales de las representaciones del tablero. Dada una entrada X ,

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

W^Q, W^K, W^V son matrices de pesos entrenables, y d_k representa la dimensión de las claves, el cual puede variar entre configuraciones de arquitectura. Este mecanismo se replica en cada una de las h cabezas de atención, permitiendo capturar distintas relaciones entre las piezas. El resultado se concatena y se proyecta linealmente de vuelta a la dimensión original de la entrada.

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Donde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, W^O es una matriz de pesos entrenables y h es el número de cabezas de atención.

5.1.4. Modelado del aprendizaje por refuerzo

El Aprendizaje por Refuerzo (RL) es un paradigma de la inteligencia artificial en el que un agente aprende a tomar decisiones secuenciales mediante la interacción con un entorno. En cada paso t , el agente observa el estado $s_t \in \mathcal{S}$ del entorno, realiza una acción $a_t \in \mathcal{A}$ según su política $\pi(a_t|s_t)$ y recibe una recompensa $R_{t+1} \in \mathbb{R}$ y una transición al siguiente estado

s_{t+1} .

El objetivo del agente es maximizar el retorno esperado $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$, donde $\gamma \in [0, 1)$ representa el factor de descuento.

Por su parte, los métodos de gradiente de política (*Policy Gradient*) son una aproximación que optimiza directamente los parámetros θ de una política $\pi_{\theta}(a_t|s_t)$ por ascenso de gradiente, con el fin de maximizar la recompensa acumulada esperada $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$ sobre trayectorias τ . Mediante el teorema del gradiente de política, el gradiente de la función objetivo se formula como la esperanza analítica:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t, a_t \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Psi_t]$$

Dependiendo del método, Ψ_t puede tomar varias formas, pero actúa como un estimador de la calidad de la decisión (como el retorno G_t o la función de ventaja $A(s_t, a_t)$), lo que orienta las actualizaciones de la política sin requerir un modelado explícito de la dinámica del entorno.

El problema del ajedrez 5×5 se modela a través de una formulación clásica de Proceso de Decisión de Markov (MDP). En cada paso t , el agente observa el estado actual s_t , elige una acción estocástica basada en una política parametrizada $\pi_{\theta}(a_t|s_t)$ y pasa a un nuevo estado s_{t+1} . La iteración prosigue hasta el final de la trayectoria (fin de la partida), momento en el cual el agente interactúa con el entorno recibiendo una recompensa terminal (victoria, derrota o empate).

A diferencia de enfoques como MCTS que optimizan una pérdida de entropía cruzada mediante búsqueda explícita, este trabajo emplea una red neuronal de política y valor conjunta sin simulación del futuro. La red se optimiza directamente utilizando la *clipped surrogate objective function* de Proximal Policy Optimization^[16]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

donde $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ es el cociente de probabilidades, $\hat{A}_t$ es la estimación de la ventaja y ϵ es el hiperparámetro de recorte (clipping). La red también minimiza el error cuadrático medio en la estimación del valor $V_{\theta}(s_t)$. La función de pérdida conjunta total es:

$$\mathcal{L} = -L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$$

Diseño de la función de recompensa

El Aprendizaje por Refuerzo es muy susceptible al fenómeno de *reward hacking*^[17], donde el agente aprende a explotar lagunas en la función de recompensa para maximizar su puntuación sin alcanzar el objetivo real del problema. Por ejemplo, si se otorgara una recompensa positiva por cada captura de material, el agente podría aprender a retrasar el jaque mate indefinidamente para “farmear” piezas del oponente, o tomar decisiones subóptimas como capturar un peón cuando se encuentre frente a una situación peligrosa.

Para evitar este comportamiento subóptimo, se define una función de recompensa dispersa R_t , que únicamente emite una señal al alcanzar un estado terminal final, denominado T :

$$R_t = \begin{cases} 1 & \text{si el estado } T \text{ es victoria} \\ -1 & \text{si el estado } T \text{ es derrota} \\ 0 & \text{si el estado } T \text{ es empate o no terminal} \end{cases}$$

Esta escasez de señal dificulta las fases iniciales del entrenamiento, lo que justifica la necesidad de técnicas avanzadas o inicialización supervisada para guiar al modelo en las etapas tempranas para acelerar la convergencia de PPO.

5.2. Marco práctico

5.2.1. Codificación del espacio de estados (input)

Para evaluar la Hipótesis 1, se instrumentarán y compararán dos representaciones del tablero de ajedrez 5x5, analizando su impacto en la asimilación espacial del modelo Transformer.

- **Representación plana simple (byte array):** El tablero se representa como un vector unidimensional de 27 *bytes* (índices 0-24), correspondientes a las 25 casillas del tablero de Minichess. Cada casilla almacena un número entero que identifica unívocamente una de las 12 piezas posibles (6 blancas y 6 negras) o el estado de casilla vacía. Adicionalmente se almacenan el número de repeticiones y el número de jugadas sin capturas / jaques en los 2 bytes finales. Esta representación es minimalista pero computacionalmente eficiente, y delega en el Transformer la responsabilidad de deducir la geometría 2D del tablero.
- **Representación espacial (estilo AlphaZero):** El tablero se expande en un tensor multicanal de dimensiones $C \times 5 \times 5$. Cada canal C está asignado a un tipo de pieza por color y actúa como un plano binario indicando su presencia o ausencia. Esta codificación facilita el reconocimiento de patrones espaciales a través de sesgos inductivos explícitos. Se añaden, además, canales binarios adicionales para variables de estado globales como el jugador al que le toca mover y los límites del half-move clock.

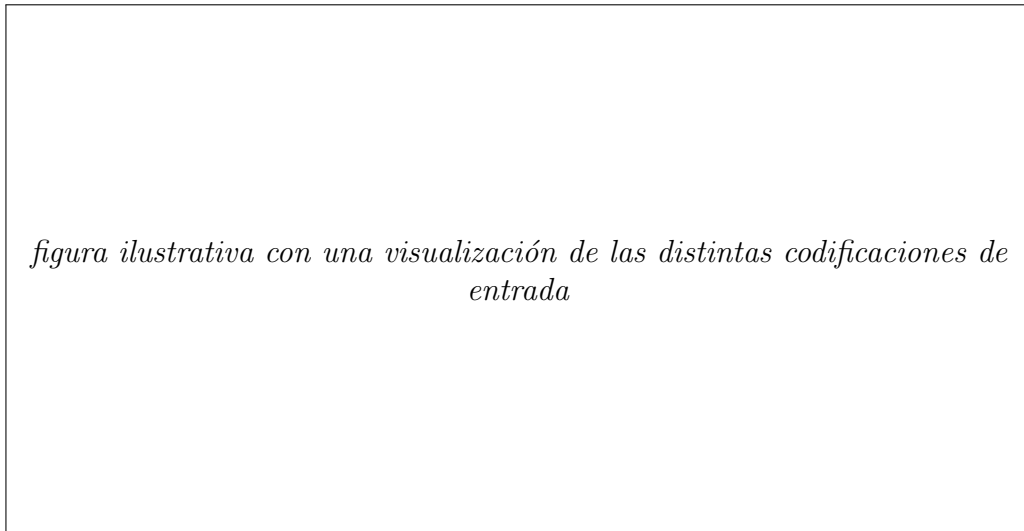


Figura 2: Visualización de codificaciones de entrada. De izquierda a derecha: a, b, c...

5.2.2. Codificación del espacio de acciones (output)

De manera análoga, la forma en que la red proyecta sus predicciones altera drásticamente la topología del espacio de optimización. Se implementarán las siguientes cabezas de salida:

- **Política plana discreta:** Consiste en un vector de salida lineal que mapea todas las posibles transiciones legales del juego. En un tablero 5x5, considerando que una pieza no puede moverse a su propia casilla origen, existen $5^2 \times (5^2 - 1) = 600$ movimientos posicionales puros. Adicionalmente, las coronaciones de peón requieren un modelado independiente.

En Minichess, los peones en las columnas exteriores (a, e) tienen 2 opciones al coronar (avance o captura simple), mientras que los peones en columnas centrales (b, c, d) tienen 3. Esto resulta en $(2 + 3 + 3 + 3 + 2) = 13$ movimientos de coronación. Multiplicado por las 4 piezas posibles de coronación (Reina, Torre, Alfil, Caballo) y los 2 colores, se añaden 104 neuronas. La salida de la política (*policy head*) será, por tanto, un vector estricto de **704 neuronas**.

- **Política factorizada (origen-destino) como tarea auxiliar:** Esta codificación no sustituye a la política discreta plana principal de 704 salidas; añade una cabeza auxiliar para informar más del gradiente de pérdida durante el entrenamiento. Se compone de tres cabezas complementarias: una para la casilla de origen del movimiento (25 salidas), otra para la casilla de destino (25 salidas) y una tercera para predecir la clase de coronación (9 salidas). La pérdida de estas cabezas se pondera con un factor de 0.5 y se suma a la pérdida total. Esto reduce el espacio de salida de esta cabeza de 704 a 59 dimensiones en total, simplificando la tarea de optimización del gradiente. El objetivo es facilitar el aprendizaje de las representaciones del backbone de atención, y acelerar la convergencia inicial del entrenamiento proporcionando una señal de error más explícita y estructurada.
- **Cabeza de valor:** Se incluye una cabeza de valor que predice el valor esperado de la posición actual, lo que permite al modelo aprender a evaluar la calidad de las posiciones además de predecir acciones. Esta cabeza se implementa como una proyección lineal seguida de una activación tanh para acotar la salida en el rango $[-1, 1]$, donde -1 representa una derrota segura, 0 un empate y +1 una victoria segura.

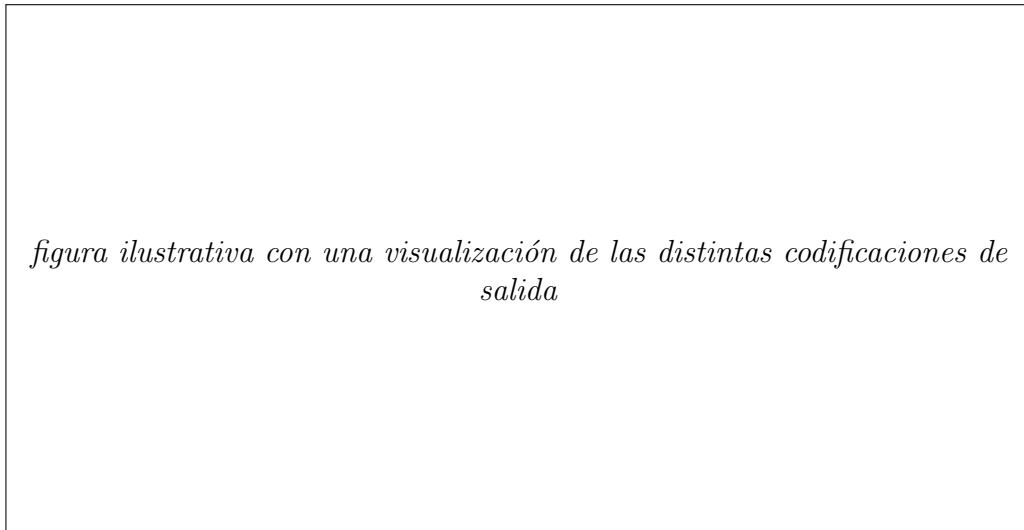


Figura 3: Visualización de codificaciones de salida. De izquierda a derecha: a, b, c...

5.2.3. Arquitectura transformer

La arquitectura neural propuesta se basa en un modelo **Transformer encoder** que procesa el estado del tablero de forma bidireccional, permitiendo que cada componente preste atención a todos los demás, y se describe a continuación:

1. **Capa de proyección de entrada:** Dependiendo de la representación seleccionada (de las mencionadas en la sección 5.2.1):
 - La representación plana simple consta de 25 casillas (que clasifican las piezas o casilla vacía), más la cuenta de repeticiones y la regla de los 50 movimientos (27 elementos en total). Adicionalmente, se concatena un token de clase **CLS**, resultando en una secuencia de 28 tokens que se proyectan a la dimensión d_k mediante una tabla de embeddings.
 - La representación espacial (estilo AlphaZero) reconstruye un tensor de dimensiones $15 \times 5 \times 5$ (12 canales de piezas, 1 canal para el jugador activo, 1 canal de repeticiones y 1 canal de halfmove clock). Cada una de las 25 casillas del tablero cuenta con un vector de 15 canales que se proyecta linealmente a la dimensión d_k . Adicionalmente, se concatena el token de clase **CLS**, resultando en una secuencia de 26 tokens.
2. **Codificación posicional:**
 - *Para la representación plana simple:* Se añade un *embedding* posicional absoluto y unidimensional de tamaño 28, asignando un vector aprendido a cada posición de la secuencia.
 - *Para la representación espacial:* Se inyecta información geométrica explícita en 2D. Se definen dos tablas de embeddings aprendidas: **row_pos_embed** de tamaño (5, D) (para las filas) y **col_pos_embed** de tamaño (5, D) (para las columnas). Para cada una de las 25 posiciones del tablero en la coordenada (y, x) , su codificación posicional es la suma de los embeddings de su fila y su columna. Para el token CLS se concatena un parámetro aprendido independiente **cls_pos_embed**.
3. **Pila de bloques encoder:** El tensor resultante se procesa a través de una serie de L capas de codificación con h cabezas. Cada capa contiene:
 - Un mecanismo auto-atención multicabeza, sin causal-masking. Esencialmente un transformer encoder como el que se podría encontrar en modelos como BERT.
 - Conexiones residuales de inicio a fin y normalización, utilizando **RMSNorm** y la convención *pre-norm*. Esta ñultima aplica normalización antes de cada subcapa de la forma $z = x + \text{sublayer}(\text{normalization}(x))$, donde *sublayer* es una de las L capas encoder, mejorando la estabilidad del entrenamiento^[18]. Esta normalización se utiliza en transformers modernos debido a que reduce la complejidad computacional frente a alternativas como LayerNorm^[19].
 - Una capa *Feed-Forward Network* o *Multi-Layer Perceptron* (MLP), compuesta de la forma:

$$\text{FFN}(x) := FC_{\text{expand}}(x) \rightarrow \text{GELU} \rightarrow FC_{\text{project}}(x) \rightarrow \text{Dropout}$$

4. **Cabezas de salida:** Tras la última normalización RMS, el estado se divide, extrayendo

el tensor del *CLS token* y aislando los 25 vectores resultantes del estado final de las iteraciones de atención sobre las celdas del tablero. A continuación se bifurca en los siguientes procesos:

- **Cabeza de política orientada a relaciones (*Matrix policy head*):** Proyecta cada uno de los 25 *tokens* de las casillas paralelamente evaluando si estas funcionan como origen (*proj_from*) y como destino (*proj_to*) mediante MLPs. A continuación realiza un producto interno espacial (*batched matmul*) entre estas incrustaciones elaborando una matriz de correlación 25×25 . Paralelamente, se obtienen otros 104 *logits* del tensor *CLS* que reflejan opciones de coronación, y se concatenan a los 600 movimientos ordinarios. Obtenemos por lo tanto, una distribución de 704 *acciones*.
 - **Cabeza de valor (*Value head*):** Procesa únicamente el *token CLS* final en un *MLP* denso (Expansión Lineal, GELU, Dropout, Reducción de dim a escalar, Tanh) que acota la salida en el intervalo $[-1, 1]$.
 - **Cabezas auxiliares de política factorizada:** Estas cabezas no se activan siempre. Se trata de las cabezas mencionada en la sección 5.2.2. Decodifican las representaciones del tablero y de CLS mediante tres proyecciones independientes en logits de casilla de origen (25), destino (25) y tipo de coronación (9). Estas cabezas se evalúan únicamente durante el entrenamiento supervisado, con propósitos de mejora de la señal de error.
5. **Funciones de Pérdida combinada:** Durante el entrenamiento supervisado se minimiza una función de pérdida multiobjetivo. Para las variantes sin pérdida factorizada auxiliar, la pérdida total $\mathcal{L}$ es la suma directa de la pérdida de la política principal $\mathcal{L}_p$ (entropía cruzada con logits), y la pérdida de valor, $\mathcal{L}_v$ (error cuadrático medio):

$$\mathcal{L} = \mathcal{L}_p + \mathcal{L}_v \quad (1)$$

Para las variantes con pérdida factorizada auxiliar, se añade la pérdida de la cabeza auxiliar factorizada, $\mathcal{L}_{\text{aux}}$ (entropía cruzada), escalada por un factor de regularización de 0,5 para informar mejor al gradiente sin eclipsar la política principal:

$$\mathcal{L} = \mathcal{L}_p + \mathcal{L}_v + 0,5 \cdot \mathcal{L}_{\text{aux}} \quad (2)$$

Las formulaciones exactas de cada componente de pérdida se detallan en el Anexo A.4.

La figura 4 ilustra la arquitectura propuesta, mostrando el flujo de datos desde la entrada de la representación hasta las salidas de las cabezas de política y valor.

5.2.4. Algoritmo de RL: Proximal Policy Optimization (PPO)

Cabe preguntarse: ¿por qué este algoritmo en lugar del MCTS usado en enfoques como AlphaGO o AlphaZero? Clásicamente, metodologías como MCTS y *Expert Iteration* actúan más como planificadores a futuro que como RL propiamente dicho: descubren futuros hipotéticos mediante búsqueda por fuerza bruta al contar con un modelo perfecto del entorno. En ellos, la red neuronal actúa como una memoria supervisada que aproxima las probabilidades de visita ya descubiertas por la búsqueda. Estos sistemas no están aprendiendo *per se*

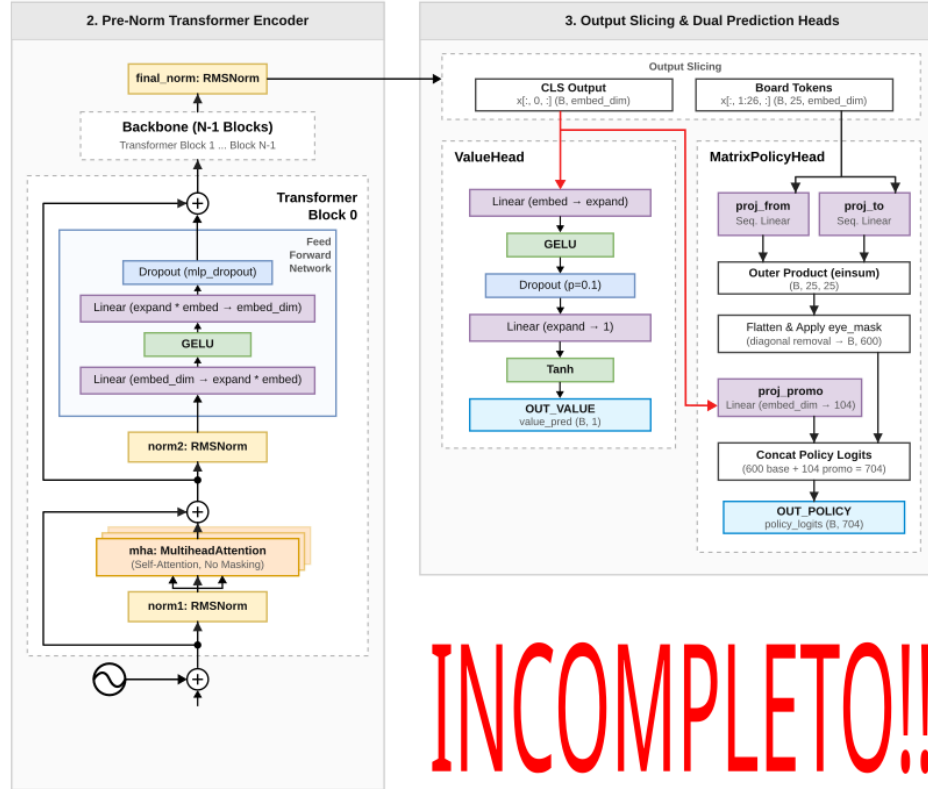


Figura 4: Diagrama de la arquitectura utilizada transformer propuesta

(algo que se obtiene de la experiencia, y por tanto del pasado), sino que están planificando (algo que se obtiene de la previsión del futuro, y por tanto de la simulación, el futuro).

Por el contrario, Proximal Policy Optimization (PPO) es un algoritmo de aprendizaje por refuerzo puro (*Model-Free*). La red neuronal carece de un mecanismo explícito de previsión del futuro mediante un árbol con estados; es ella, iterativamente a través métodos Actor-Crítico y puramente por descenso de gradiente, la que asimila las estrategias óptimas guiada por ensayo y error. Además, al requerirse exactamente una única inferencia por tablero en cada estado, los tableros / estados pueden agruparse en (*batches*) mayores, maximizando la ocupación de GPU.

Mecánica de optimización y estimación de ventaja

El algoritmo PPO actualiza los pesos para evitar colapsar debido a cambios demasiado severos de la política por una actualización errónea. Esta es la función fundamental del hiperparámetro de recorte ϵ . Este asegura que la actualización no destruya la política existente, al penalizar las actualizaciones estocásticas grandes que diverjan significativamente de la política anterior empleada para recolectar el episodio en cuestión.

Para balancear correctamente el sesgo y la varianza a la hora de determinar lo buena que es la acción tomada respecto a la acción promedio en el estado actual, se emplea la Estimación Generalizada de Ventaja (GAE - *Generalized Advantage Estimation*). El algoritmo se puede sintetizar en el pseudocódigo del Algoritmo 1, derivado del pseudocódigo original de Shulman et al. (2017)^[16]:

Algorithm 1 Proximal Policy Optimization (PPO-Clip)

```

1: Inicializar parámetros de política y valor  $\theta$ 
2: for iteración  $i = 1, 2, \dots$  do
3:   Ejecutar política parametrizada  $\pi_{\theta_{old}}$  en  $N$  entornos paralelos durante  $T$  pasos tem-
     porales
4:   Calcular las estimaciones de ventaja  $\hat{A}_t$  mediante GAE, en base al valor de  $V_\theta(s_t)$  y
     las recompensas obtenidas
5:   for época  $k = 1, \dots, K$  do
6:     for cada mini-batch do
7:       Calcular el cociente  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 
8:       Calcular pérdida recortada:  $L^{CLIP} \leftarrow \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$ 
9:       Calcular pérdida de valor:  $L^{VF} \leftarrow (v_\theta(s_t) - V_t^{target})^2$ 
10:      Calcular pérdida de entropía:  $L^{ENT} \leftarrow H(\pi_\theta(s_t))$ 
11:      Retropropagar para minimizar  $L^{CLIP} + c_1 L^{VF} - c_2 L^{ENT}$ 
12:    end for
13:  end for
14:   $\theta_{old} \leftarrow \theta$ 
15: end for
    
```

En este pseudocódigo, se deben denotar varios aspectos:

- Tanto V_θ como π_θ están parametrizadas por el mismo θ , dado que ambas funciones comparten el backbone de la arquitectura, pero cada una se predice en una de sus cabezas.
- c_1 y c_2 son coeficientes de ponderación para la pérdida de valor y la pérdida de entropía, respectivamente. La función de entropía $H(\pi_\theta(s_t))$ se incluye para fomentar la exploración durante el entrenamiento, y el valor objetivo V_t^{target} se calcula utilizando las recompensas obtenidas y las estimaciones de valor futuro.
- La ventaja en el pseudocódigo anterior se calcula utilizando GAE, y se controla mediante el hiperparámetro λ que ajusta el sesgo-varianza de la estimación. Un valor de λ cercano a 1 favorece una estimación de ventaja con bajo sesgo pero alta varianza (aproxima Monte Carlo), mientras que un valor cercano a 0 produce una estimación con alto sesgo pero baja varianza que aproxima *TD-learning*.

Este hiperparámetro no está presente explícitamente en el pseudocódigo, pero se utiliza en la función de cálculo de $\hat{A}_t$ para determinar cómo se combinan las recompensas futuras y las estimaciones de valor para obtener una ventaja más estable y eficiente. El cálculo de la ventaja $\hat{A}_t$ mediante GAE se realiza utilizando la siguiente fórmula:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t)$$

5.2.5. Descripción del conjunto de datos

Para el entrenamiento en la fase de aprendizaje supervisado, se ha optado por utilizar exclusivamente un conjunto de datos sintéticos generado con una profundidad de búsqueda de 4 jugadas por `fairy-stockfish`, descartando por completo el uso de mezclas con evaluaciones de profundidades inferiores ($D2$ o $D3$). Este conjunto de datos definitivo cuenta con un volumen total de 60,030,000 posiciones, el cual ha sido procesado mediante una división estricta de un 97 % para el conjunto de entrenamiento (58,234,050 posiciones) y un 3 % reservado para el conjunto de validación (1,795,950 posiciones).

La elección de entrenar el modelo únicamente con datos analizados a profundidad 4 responde a la necesidad de capturar la complejidad, sin utilizar algoritmos de búsqueda o planificación como MCTS en la fase de RL. El agente debe aprender a modelar la búsqueda como una función. Modelar el comportamiento de un motor clásico a este nivel obliga a la arquitectura a ir más allá de heurísticas simples. El conjunto de datos presenta cierta complejidad debido a dos factores principales:

- **Aislamiento de posiciones quiescentes:** El generador de datos de `fairy-stockfish` ha filtrado y eliminado sistemáticamente las posiciones tácticas puras (tales como capturas inmediatas de material o movimientos bajo jaques forzados). Este filtrado se realiza habitualmente en entrenamiento de redes neuronales para motores de ajedrez con búsqueda, ya que obliga a la red a desarrollar una comprensión posicional, dejando los cálculos tácticos para el algoritmo de búsqueda. Las posiciones resultantes se conocen como *quiescent positions* y son aquellas donde no hay amenazas inmediatas de capturas, jaques, etc. Esto garantiza que el Transformer no aprenda respuestas reactivas simples, sino que se vea obligado a internalizar patrones posicionales complejos o a emular conceptualmente la propia idea de búsqueda en profundidad bajo condiciones de recompensas dispersas.
- **Filtrado de posiciones repetidas:** Se ha aplicado un proceso de deduplicación sobre el archivo crudo para eliminar estados idénticos del tablero, previniendo memorización y contaminación de los conjuntos de validación o test.

Asimismo, el conjunto de datos cuenta con particularidades estructurales importantes:

- **Restricción en las coronaciones:** A diferencia del modelo teórico que contempla múltiples opciones de coronación, las coronaciones registradas se limitan a la dama por una restricción de la librería de generación de datos. Prácticamente, esta suele ser la jugada óptima y la red sigue permitiendo la subcoronación en la posterior fase de RL.
- **Falta de información de repetición en FEN:** El formato FEN representa de manera estática el estado físico del tablero, el jugador activo, los derechos de enroque y la regla de los 50 movimientos. Sin embargo, carece del historial de posiciones pasadas de la partida, haciendo imposible determinar a partir de un solo FEN si se ha producido una repetición de la posición (regla de la triple repetición).
- **Peculiaridad de la señal de resultado:** El campo `result` de las muestras de entrenamiento está expresado desde la perspectiva del jugador al que le toca mover. De este modo, un valor de 1 indica la victoria para el jugador activo, un valor de -1 indica su derrota, y un 0 indica tablas, a diferencia de una codificación absoluta donde el signo se asocia fijamente a un color.
- **Métricas de distribución y balance:** Sobre una muestra aleatoria representativa

de 10,005,000 posiciones analizadas en detalle, se registra una clara predominancia de tablas con un ratio del 54,31 % (5,433,596 posiciones). El resto del conjunto refleja una ligera ventaja para las blancas, obteniendo un 23,53 % de victorias (2,354,634 posiciones) frente al 22,16 % de las negras (2,216,770 posiciones). Un desglose completo de estas estadísticas se incluye en el Anexo A.2.

```
fen rnb1k/1p1qp/pPp2/P1PP1/RNBQK w - - 0 4
move d1e2
score 272
ply 6
result 0
e
fen rnb1k/1p1qp/pP3/P1PNQ/R1B1K b - - 0 5
move b5c3
score 31
ply 9
result 0
[...]
```

Figura 5: Fragmento de ejemplo del conjunto de datos de entrenamiento. Cada muestra está delimitada por la letra **e** y contiene el estado en notación FEN, el movimiento seleccionado, la puntuación heurística del motor, el número de ply y el resultado relativo.

5.2.6. Herramientas y tecnologías empleadas

El desarrollo práctico e investigativo del proyecto se implementará utilizando las siguientes herramientas de desarrollo de software:

- **Lenguaje de programación:** Python 3.12+ por su extenso soporte y bibliotecas nativas de Deep Learning.
- **Framework de Deep Learning:** PyTorch para el diseño de redes neuronales, cálculo automático de gradientes y aceleración en GPU.
- **Motor de ajedrez:** Se emplea **Fairy-Stockfish**, una versión de Stockfish para variantes personalizadas, a través de la API **pyffish** de Python y a través de su CLI propia para generación de datos para la fase de aprendizaje supervisado. Este motor proporciona: (a) validación robusta y rápida de la legalidad de movimientos y reglas de la variante Gardner 5×5 , y (b) evaluaciones heurísticas adecuadas para inicializar la red mediante aprendizaje supervisado.
- **Seguimiento de experimentos:** **tensorboard** se utilizó para la visualización en tiempo real del progreso de entrenamiento, y métricas como la pérdida multiobjetivo, entropía, normas de gradientes, etc.
- **Reproducibilidad, dependencias y manejo de módulos en Python:** **uv** para la gestión de entornos virtuales y dependencias, asegurando la portabilidad y reproducibilidad del código (**uv** permite generar archivos de requisitos específicos y fijar una versión de Python).

- **Ajuste de hiperparámetros:** Optuna se utilizó para la optimización de hiperparámetros fundamentales como el learning rate o parámetros del optimizador.
- **Testing:** Aunque el objetivo central de este trabajo no es el desarrollo de software, si no la obtención de conclusiones acerca de las hipótesis planteadas, se usó `pytest` como framework de testing para desarrollar tests de componentes como los bindings de pyffish en python, el entorno de RL, o el funcionamiento de la implementación de PPO.
- **Tests estadísticos:** Se utilizaron `scipy` y `statsmodels` para realizar tests estadísticos sobre los resultados obtenidos en los experimentos.

5.3. Diseño experimental

5.3.1. Diseño experimental para hipótesis 2

Para demostrar el efecto de distintas representaciones del input y de output, se ha llevado a cabo el siguiente experimento:

1. **Fijar una “receta” de red neuronal y de entrenamiento:** hiperparámetros de la red como la dimension de embeddings o bloques transformer + hiperparámetros de entrenamiento como LR, batch size, epochs y parametros de AdamW (detalles en el anexo A.1 y tabla 5)
2. **Se definen 4 versiones:** 1 representa al modelo base y las 3 restantes, ablaciones sobre este. Las 4 versiones son:
 - (Baseline) Codificación de input simple. Cabezas simples: política y valor.
 - Codificación de input simple. Cabezas factorizadas: política, valor y política factorizada.
 - Codificación de input bidimensional (estilo AlphaZero). Cabezas simples: política y valor.
 - Codificación de input bidimensional (estilo AlphaZero). Cabezas factorizadas: política, valor y política factorizada.
3. **Seleccionar varias semillas fijas (10).**
4. **Para cada semilla, repetir los siguientes pasos para eliminar el factor de aleatoriedad:**
 - Para cada versión de las mencionadas anteriormente:
 - a) Ejecutar un entrenamiento completo, manteniendo todos los hiperparámetros fijos.
 - b) Registrar y guardar resultados: loss de train y validation para cada cabeza, accuracy en train y validation, checkpoint del mejor modelo, numero de parámetros, FLOPs estimados...
 - c) Luego, ejecutar este checkpoint contra el conjunto de test para obtener los resultados finales para estudiar el efecto de cada versión sobre el modelo
5. **Una vez obtenidos los resultados de test para las semillas, utilizar una serie de tests estadísticos para confirmar / desmentir si la hipótesis es estadísti-**

camente significativa. Se debe comprobar que la hipótesis se cumple teniendo en cuenta la variabilidad de los resultados entre semillas. Para ello se debe decidir entre tests paramétricos o no paramétricos, demostrando la distribución de los datos, y a continuación realizar una prueba estadística para confirmar / desmentir la hipótesis. **Outputs esperados:** tabla con resultados medios y desviación estándar de cada versión, gráficas que muestren la evolución media y varianza de loss y accuracy (media = línea, varianza = sombreado), y resultados de los tests estadísticos.

6. **Para testear más allá de la accuracy o loss, hay que enfrentar los modelos entre ellos:** la capacidad de predicción no tiene por qué correlacionarse con la capacidad de juego. Por ello, se llevará a cabo lo siguiente:
 - a) Elegir una sola semilla. Usar esta para todos los modelos.
 - b) A las 4 variantes anteriormente mencionadas, añadir 3 modelos más que servirán de comparativa: un MLP base, un agente heurístico, y un Agente aleatorio.
 - c) Para cada par de modelos, ejecutar un torneo de 100 partidas (50 por cada color).
 - d) Registrar resultados y calcular winrate, ELO estimado, entropía de la política, etc.
 - e) Analizar los resultados. Extraer conclusiones sobre la capacidad de juego entre modelos, y con respecto a los resultados de test obtenidos en el paso anterior.
 - f) Output buscado: una matriz de tamaño $N \times N$ (N = número de modelos) con los resultados de winrate entre cada par de modelos, y una tabla con el ELO estimado de cada modelo.

5.3.2. Diseño experimental para la hipótesis 3

Para demostrar el efecto de la inicialización supervisada sobre la convergencia y la fuerza de juego final en el bucle de Aprendizaje por Refuerzo, se ha diseñado e instrumentado el siguiente experimento:

1. **Fijar una “receta” de red neuronal y de algoritmo de optimización:** Se seleccionará la arquitectura de red ganadora surgida del estudio de ablación de la Hipótesis 2 (la configuración `spatial_nofact`). Asimismo, se congelarán los hiperparámetros de PPO y del buffer de recolección, incluyendo la tasa de aprendizaje (lr), **batch** size, el horizonte temporal de *rollouts* (T), entre otros (detalles específicos en la sección A.7).
2. **Definimos las 2 variantes a comparar:** El experimento contrasta directamente el impacto del punto de partida ponderado de los parámetros:
 - **Agente tabula rasa:** El modelo se inicializa con pesos aleatorios siguiendo la convención estándar, obligando al bucle de RL a explorar el espacio de estados sin conocimiento previo.
 - **Agente pre-entrenado:** Los pesos del backbone de atención y de las cabezas duales se inicializan cargando el checkpoint óptimo derivado de la fase de Aprendizaje Supervisado.
3. **Seleccionar varias semillas fijas (5):** Para mitigar la alta varianza inherente al muestreo estocástico en entornos de recompensa dispersa, y para asegurar la reproducibilidad científica, se fijan 5 semillas independientes de inicialización de los entornos y del generador de números aleatorios.

4. Para cada semilla, ejecutar el bucle adversarial completo:

- Para cada una de las 2 variantes (Tabula rasa y pre-entrenado):
 - a) Desplegar una suite de N entornos de Minichess vectorizados en paralelo.
 - b) Ejecutar el entrenamiento de PPO durante un horizonte fijo de iteraciones / episodios.
 - c) **Estrategia de pool de oponentes:** Siguiendo el método de optimización continua de AlphaZero, los juegos de *self-play* se generan utilizando directamente los parámetros de la red en su estado más reciente^[4], omitiendo pasos de evaluación explícitos o filtros de corte de rendimiento estilo AlphaGo Zero. No obstante, para prevenir el olvido catastrófico y el sobreajuste a la política inmediata anterior, se implementa un buffer mixto de emparejamiento con un ratio de 80/20:
 - El **80 %** de los entornos vectorizados jugarán contra la versión de la red continuamente actualizada (*self-play* puro).
 - El **20 %** restante de los entornos se emparejará contra un checkpoint aleatorio extraído del histórico del propio agente. Este almacena una copia en disco de la red de forma periódica cada K iteraciones de PPO.
 - d) Durante la recolección, las acciones se seleccionarán mediante muestreo estocástico directo a partir de las probabilidades obtenidas por la cabeza de política una vez aplicada la máscara de movimientos ilegales.
 - e) Registrar y guardar los datos internos del bucle: pérdida de la política (*Policy Loss*), pérdida del crítico (*Value Loss*), entropía de la acción y recompensa media del buffer.

Dado que la recompensa interna del buffer es una métrica móvil relativa al nivel del pool de oponentes, la verdadera fuerza de juego se monitorizará mediante evaluaciones externas estáticas. Cada 5 iteraciones de PPO, se pausará el entrenamiento de forma síncrona y el modelo congelado disputará un torneo cerrado de 40 partidas contra un agente aleatorio y el agente heurístico. A partir de los resultados de este torneo, se calculará la tasa de victoria y el ELO estimado del agente.

5. **Validación estadística de la convergencia:** Una vez finalizadas las ejecuciones de las 5 semillas, se extraerán los valores finales de ELO y las tasas de victoria del torneo. Se aplicarán tests estadísticos con el fin de rechazar matemáticamente la hipótesis nula de igualdad de medias y certificar que la aceleración del modelo preentrenado es estadísticamente significativa ($\alpha = 0,05$). **Outputs esperados:** Gráfico de líneas temporal con la evolución del ELO estimado frente al oponente heurístico (donde la línea sólida representa la media paramétrica de las semillas y la región sombreada su desviación estándar $\pm\sigma$), y una tabla comparativa que contraste el ELO máximo e iteración de estabilización entre ambas variantes. Gráficas de recompensa media, con marcadores que indican los puntos en los que acaba un episodio (y por tanto puede haber un cambio de oponentes, y una fluctuación en la recompensa media).

6. Planificación y seguimiento

La siguiente planificación es una aproximación orientada a la experimentación rápida. Para minimizar la fricción en el desarrollo de los experimentos y la lógica interna del ajedrez Gardner esta se delega a `pyffish`, y del mismo modo, en la medida de lo posible los demás componentes se delegan también. Esta decisión acelera las fases iniciales del proyecto y permite además obtener datos de buena calidad para el preentrenamiento de los modelos.

Las distintas secciones no cuentan con una duración fija. A continuación se presenta el plan de trabajo, dividido en fases:

Fase 0: Investigación previa y documentación inicial

- Investigación del estado del arte en RL y Transformers aplicados al ajedrez.
- Consolidación de objetivos generales y específicos. Creación del marco teórico y práctico para el desarrollo del TFG.
- Diseño de arquitectura de red neuronal, y algoritmo de aprendizaje por refuerzo. Diseño de metodología de experimentación e hipótesis.

Fase 1: Preparación del entorno y generación de datos sintéticos

- Integración de `pyffish` para la configuración de la variante `gardner` y validación de movimientos.
- Desarrollo y ejecución de scripts de *self-play* basados en evaluaciones a profundidad controlada de Fairy-Stockfish para construir el dataset de supervisado inicial.

Fase 2: Arquitectura y preentrenamiento (supervisado)

- Implementación del modelo Transformer Encoder, dataloaders, parsing de datos.
- Ajuste básico de hiperparámetros fundamentales.
- Ejecución de experimentos de aprendizaje supervisado utilizando los datasets generados, evaluando la velocidad de convergencia y capacidad de las distintas representaciones de tablero y acción.

Fase 3: Aprendizaje por refuerzo (pipeline de self-play)

- Construcción del entorno vectorizado para muestreo paralelo guiado por las predicciones de política y valor del Transformer.
- Implementación o integración del algoritmo Proximal Policy Optimization (PPO).
- Testing de funcionamiento correcto del entorno, recolección de rollouts y funcionamiento de PPO.

Fase 4: Evaluación y generación de métricas

- Ejecución de torneos automatizados entre agentes de control.
- Extracción y visualización de las métricas de rendimiento para la validación de las hipótesis planteadas.
- Interpretación de resultados. Tests estadísticos y generación de gráficas.

Fase 5: Documentación y conclusiones

- Documentación concurrente al desarrollo. Se diseñan los experimentos, programan, ejecutan y obtienen resultados, y se documenta el proceso previamente a cada uno de estos pasos. Por tanto, no es una fase que se aborde al final, sino que es un proceso continuo a lo largo de todo el proyecto.
 - Consolidación de los resultados en el documento, análisis crítico de las gráficas resultantes y preparación de la defensa.
-

7. Principales aportaciones

7.1. Efectos de la representación sobre la eficiencia y calidad del aprendizaje

Para contrastar la **hipótesis 2**, que plantea que las representaciones estructuradas del tablero y del espacio de acciones mejoran la eficiencia del aprendizaje en comparación con las representaciones planas, se ha llevado a cabo una evaluación sistemática y un análisis estadístico sobre las 4 variantes de arquitectura bajo 10 semillas aleatorias independientes. Los detalles del modelo utilizado e hiperparámetros fijados pueden consultarse en el anexo A.1. Cabe destacar que este modelo es relativamente pequeño (dimensión de embedding de 64, 3 capas transformer), y por tanto ve su capacidad limitada. Las conclusiones aquí obtenidas se desarrollan sobre este, pero se realizaron pruebas idénticas sobre un modelo ligeramente mayor (dimensión de embedding de 128, 5 capas transformer), que obtuvieron conclusiones en la misma dirección, con resultados mejores, lo cual apunta a que el escalado de estos modelos permite mejorar su capacidad. Estas pueden consultarse en el anexo A.3.

7.1.1. Evaluación sobre el conjunto de test

Los modelos se evaluaron sobre un conjunto de test holdout que contiene 299,325 posiciones de Gardner Minichess extraídas de forma independiente de las trayectorias de entrenamiento. Se reportan métricas tanto para la cabeza de política (predicción de movimientos) como para la cabeza de valor (predicción de resultados).

Métricas para cabeza de política (predicción de movimientos):

- **Precision@1**: porcentaje de acierto del modelo al seleccionar la jugada con mayor probabilidad en comparación con la jugada óptima.
- **Precision@3** y **Precision@5**: porcentaje en la que la jugada del conjunto de test se encuentra contenida dentro del conjunto de las 3 o 5 predicciones con mayor probabilidad del modelo.

Cabeza de valor (predicción del resultado de la partida):

- **Result accuracy**: Exactitud en la clasificación del resultado esperado de la posición tras redondear la salida continua a la clase discreta más cercana en $\{-1, 0, 1\}$ (derrota, tablas, victoria).

- **Value MSE:** Error cuadrático medio de la predicción de la cabeza de valor con respecto al resultado final real de la partida.

La Tabla 1 presenta los resultados de test consolidados promediando el rendimiento final de las 10 semillas independientes para cada variante:

Modelo / Representación	Move Acc (P@1)	Precision@3	Precision@5
<code>simple_nofact</code> (Plano, Simple)	48,63 % $\pm$ 0,17 %	83,93 % $\pm$ 0,12 %	95,17 % $\pm$ 0,05 %
<code>simple_fact</code> (Plano, Factorizado)	47,67 % $\pm$ 0,20 %	83,19 % $\pm$ 0,16 %	94,86 % $\pm$ 0,06 %
<code>spatial_nofact</code> (2D, Simple)	48,49 % $\pm$ 0,22 %	83,81 % $\pm$ 0,19 %	95,11 % $\pm$ 0,10 %
<code>spatial_fact</code> (2D, Factorizado)	48,07 % $\pm$ 0,33 %	83,52 % $\pm$ 0,25 %	95,00 % $\pm$ 0,11 %

(a) Métricas de la cabeza de política (movimientos).

Modelo / Representación	Result Acc	Value MSE
<code>simple_nofact</code> (Plano, Simple)	71,88 % $\pm$ 0,13 %	0,2806 $\pm$ 0,0008
<code>simple_fact</code> (Plano, Factorizado)	71,68 % $\pm$ 0,18 %	0,2815 $\pm$ 0,0013
<code>spatial_nofact</code> (2D, Simple)	80,39 % $\pm$ 0,06 %	0,1546 $\pm$ 0,0008
<code>spatial_fact</code> (2D, Factorizado)	79,83 % $\pm$ 0,24 %	0,1588 $\pm$ 0,0019

(b) Métricas de la cabeza de valor (resultado).

Cadro 1: Rendimiento medio y desviación estándar de las métricas sobre el conjunto de test (10 semillas $\times$ 4 variantes).

7.1.2. Test de significatividad estadística

Con el propósito de validar de forma matemáticamente rigurosa que las diferencias observadas entre las arquitecturas de ablación no se deben a fluctuaciones aleatorias provocadas por la semilla de inicialización, se aplicó un análisis estadístico compuesto por un test de Shapiro-Wilk para comprobar la normalidad, un test ANOVA de una vía para evaluar la significatividad de las diferencias globales, y comparaciones por parejas post-hoc utilizando el test de Tukey HSD. El análisis completo, junto con las tablas de resultados numéricos de las pruebas, se expone en detalle en el Anexo A.6. A continuación se resumen las conclusiones principales derivadas de estos tests:

Predicción de Movimiento (Política): El test de Shapiro-Wilk no permite rechazar la hipótesis de normalidad ($p = 0,1272 > 0,05$), lo que valida la aplicación del test paramétrico ANOVA. El ANOVA confirma la existencia de diferencias altamente significativas entre modelos ($p = 6,92 \times 10^{-10}$), por lo que procede realizar un análisis comparativo por parejas. Las comparaciones post-hoc de Tukey HSD muestran que:

- No existen diferencias estadísticamente significativas en la exactitud del movimiento entre la representación plana y la espacial (comparación entre `simple_nofact` y `spatial_nofact`, con diferencia de $-0,14\%$ y $p_{adj} = 0,5926$).
- La factorización del espacio de acciones degrada significativamente el rendimiento de la política, reduciendo la precisión tanto en el modelo plano ($-0,96\%$, $p_{adj} = 0,0$) como en el espacial ($-0,42\%$, $p_{adj} = 0,0036$).

Predicción de Resultado (Valor): El test de Shapiro-Wilk detecta una desviación del

comportamiento estrictamente normal ($p < 0,05$), no obstante, el test ANOVA global sigue siendo altamente significativo ($p = 2,11 \times 10^{-50}$). El test post-hoc de Tukey confirma que:

- La representación espacial (2D) aporta una mejora muy significativa a la cabeza de valor. Se obtiene un incremento absoluto de la exactitud de resultado de +8,51 % en el modelo simple ($p_{adj} = 0,0$) y de +8,15 % en el factorizado ($p_{adj} = 0,0$), reduciendo el error cuadrático medio (MSE) casi a la mitad (de 0,2806 a 0,1546).
- Nuevamente, la factorización empeora significativamente el desempeño, registrando la variante sin factorizar `spatial_nofact` un rendimiento superior de +0,56 % con respecto a `spatial_fact` ($p_{adj} = 0,0$).

Los boxplots de la Figura 6 ilustran visualmente la dispersión y consistencia de las métricas en test para las 10 semillas independientes, respaldando las conclusiones de significatividad estadística obtenidas.

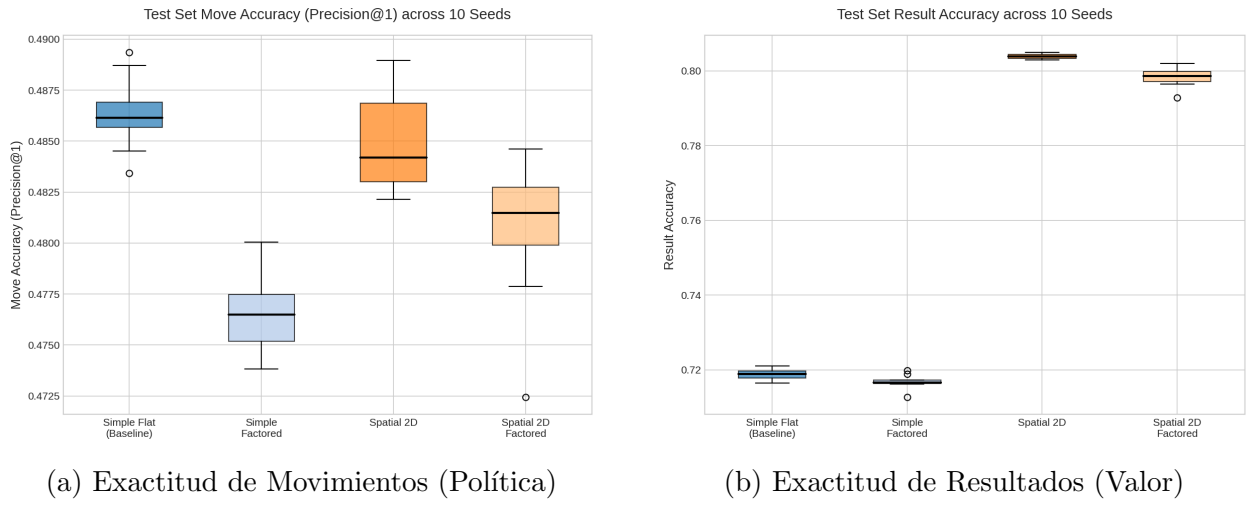
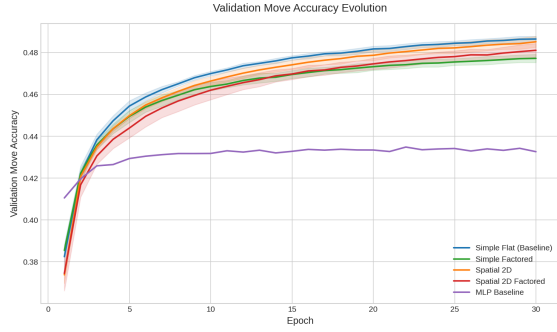


Figura 6: Distribución y variabilidad de las métricas en test sobre las 10 semillas independientes.

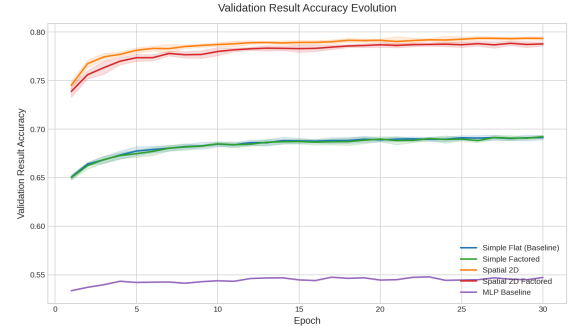
7.1.3. Evolución y curvas de aprendizaje

Para analizar el comportamiento dinámico durante el entrenamiento de 30 épocas, se promediaron las métricas de validación y entrenamiento a través de las 10 semillas. La Figura 7 muestra las curvas de evolución media (línea continua) y su desviación estándar (área sombreada). Las paradas tempranas aplicadas durante el entrenamiento se gestionan manteniendo el último estado registrado hasta el final del estudio (carry forward).

Observamos que la exactitud del movimiento las curvas de los modelos planos y espaciales sin factorizar son prácticamente paralelas e idénticas en convergencia, mientras que la exactitud de resultados muestra que el modelo espacial converge a una asíntota notablemente superior a la del modelo plano.



(a) Move Accuracy (Validation)



(b) Result Accuracy (Validation)

Figura 7: Curvas de aprendizaje promediadas con sombreado de desviación estándar.

7.1.4. Evaluación en juego real

A pesar de que la evaluación anterior sobre el conjunto de test permite medir la capacidad predictiva de los modelos en situaciones aisladas, el objetivo final del agente es interactuar con el jugador opuesto y ganar partidas reales de ajedrez, y las métricas no tienen por qué correlacionarse directamente con la capacidad de juego del agente. Con este fin, se diseñó un torneo round-robin enfrentando directamente a todos los agentes entre sí, para mostrar la capacidad de juego real.

Configuración Experimental del Torneo: El torneo se ejecutó fijando una serie de parámetros para garantizar la robustez de las comparaciones. Se disputaron 100 partidas por enfrentamiento, divididas equitativamente entre 50 partidas con blancas y 50 con negras para mitigar sesgos de color, con una única semilla (semilla 1) para todas las partidas. Se fijó un límite de 100 movimientos por partida (50 jugadas completas, en caso de alcanzar el límite se considera tablas) y se estableció una temperatura $\tau = 0,15$ para el muestreo de movimientos, y para evitar una política completamente codiciosa.

Es importante explicar el mecanismo de selección de jugadas de los agentes Transformer. En cada turno, el agente utiliza su cabeza de política para extraer las k mejores jugadas legales con mayor probabilidad, donde $k = \min(6, \text{número de jugadas legales})$. A continuación, genera los FEN correspondientes a los tableros resultantes de dichas jugadas y los introduce en lote (batch) a través de la cabeza de valor. El agente selecciona finalmente el movimiento que minimiza la predicción de valor del rival (equivalente a maximizar su propio valor debido al carácter de suma cero del juego). Esto equivale a una búsqueda de profundidad 1 en el árbol de juego, y es similar al proceso de AlphaZero, en el que la red de política se usa para reducir la anchura de la búsqueda MCT, y la red de valor se usa para reducir la profundidad.

El torneo incluye a los 4 modelos del estudio de ablación (entrenados bajo la Semilla 1), una red MLP de referencia (MLP_Baseline, con un tamaño oculto de 1024 neuronas, tasa de aprendizaje de 0.002 y entrenada con un tamaño de lote de 512), un agente de movimientos uniformemente aleatorios (RandomAgent), y un agente heurístico codicioso (HeuristicAgent) que prioriza la captura inmediata de piezas enemigas ponderadas por su valor convencional de material (Dama=9, Torre=5, Alfil/Caballo=3, Peón=1).

Estimación del Rating Elo: Los resultados del torneo se modelan mediante el sistema de Elo, que estima la habilidad relativa de los agentes basándose en los emparejamientos uno a uno. Específicamente, se ajusta un modelo de Bradley-Terry mediante estimación de máxima

verosimilitud (MLE). El modelo Bradley-Terry estima la probabilidad esperada de que el agente i gane al agente j ^[20] como:

$$P(i > j) = \frac{e^{R_i}}{e^{R_i} + e^{R_j}} = \frac{1}{1 + 10^{(R_j - R_i)/400}}$$

donde R_i y R_j son los ratings Elo de los agentes. Para calibrar la escala, se ancla el rating del agente **RandomAgent** a un valor de 1000. Los ratings Elo estimados y la entropía de la política media para cada modelo se presentan en la Tabla 2.

Agente	Arquitectura	Elo Estimado	Entropía Media
spatial_nofact	Transformer (2D, Simple)	1168.8	1.4067
spatial_fact	Transformer (2D, Factored)	1135.2	1.4230
HeuristicAgent	Heurística de Captura	1129.4	N/A
simple_fact	Transformer (Plano, Factored)	1107.8	1.2665
MLP_Baseline	MLP Multicapa Base	1007.0	1.5113
RandomAgent	Aleatorio	1000.0	N/A
simple_nofact	Transformer (Plano, Simple)	964.7	1.3037

Cadro 2: Clasificación final y Elo estimado de los agentes tras el torneo round-robin de 100 partidas.

La Figura 8 muestra las matrices de tasa de victoria resultantes, tanto considerando las tablas (wins + 0.5*draws) como ignorando las tablas (decisive games).

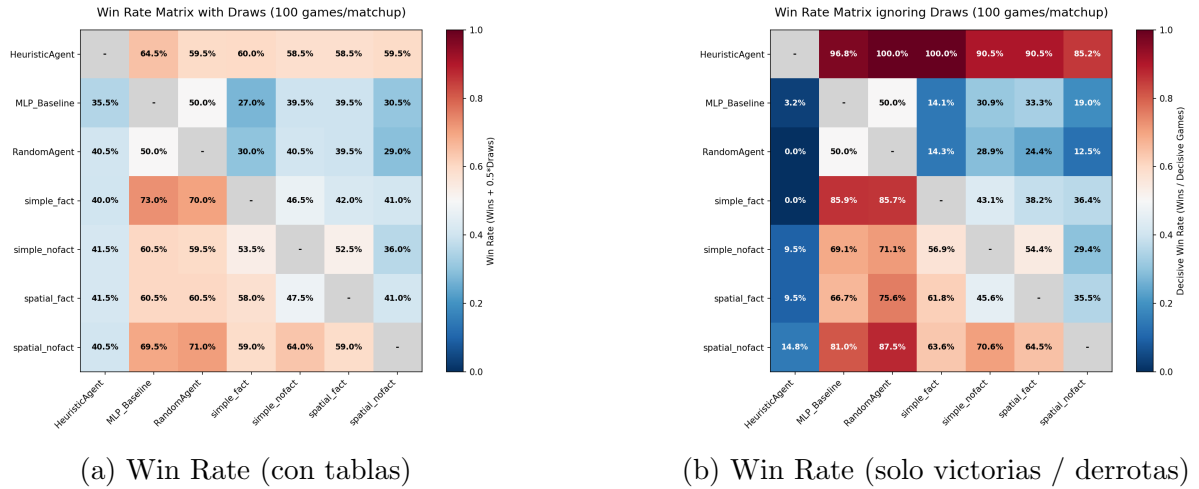


Figura 8: Matrices de winrate del torneo de 100 partidas.

7.1.5. Discusión de los resultados

Analizando los resultados obtenidos, podemos extraer una serie de conclusiones interesantes. Las más relevantes son la **confirmación de que una mejor representación ayuda al modelo**; el agente **spatial_nofact** obtiene la mejor puntuación en el torneo, con 1168.8 de Elo, superando significativamente a la versión plana **simple_nofact** (964.7 Elo, una diferencia de +204 puntos de Elo). Esta diferencia de rendimiento se alinea con el incremento del +8,51 % observado en la exactitud del valor durante el test supervisado, y con los resultados

obtenidos en las tablas de la tabla 1 y la figura 6, y también señala a que la capacidad para estimar el valor de la posición resulta en mejor capacidad de juego.

Esto demuestra que **añadir información espacial ayuda al modelo a obtener una mejor representación interna** y por tanto a tomar mejores decisiones. Esto concuerda con los resultados de estudios previos como Czech et al.^[21] y Monroe et al.^[8], que demuestran, respectivamente, que sigue existiendo cierta importancia en la ingeniería de características (especialmente en dominios específicos como el ajedrez) a pesar del poder de los Transformers, y que los sesgos inductivos (tanto a nivel de input como de codificación posicional) son potentes a la hora de mejorar el rendimiento en dominios específicos. Esto también se alinea con la intuición general de que los Transformers son muy sensibles a la representación de entrada y de salida (codificación de input y función de pérdida), como se menciona recientemente en^[22].

Por otro lado, observamos que **el uso de una cabeza extra de política no mejora el rendimiento del agente e incluso empeora**, tanto para los resultados de valor como de movimiento. Hay varias razones que pueden explicar este fenómeno:

1. Debido al diseño de la cabeza de política, ya existe un sesgo inductivo muy similar al de esta cabeza factorizada, que viene dado por el uso de la proyección a un vector de origen y destino y la posterior batch matrix multiplication, lo que puede anular su efecto sobre la representación interna.
2. La función de pérdida utilizada de valor tiene una magnitud aproximadamente 5 veces menor que la de política. Si a esta función de pérdida añadimos la función de pérdida de la política factorizada, aún reducida al multiplicar por 0.5, es posible que esto resulte en un desbalance de la pérdida, provocando que el modelo reciba más "señal" de la política y menos del valor.
3. Finalmente, para explicar el empeoramiento de la política factorizada sobre los movimientos, una posibilidad es que este se deba a la introducción de un sesgo de independencia condicional. Al forzar a la red a predecir mediante tres sub-cabezas independientes (casilla de origen, casilla de destino y tipo de coronación), se introduce un sesgo inductivo de independencia condicional en la probabilidad conjunta de la jugada. Para el aprendizaje supervisado puro, esto puede ser un lastre: a pesar de resultar más informativo, acertar una jugada de profundidad de búsqueda 4 es más difícil si la red tiene que coordinar el acierto de tres variables discretas por separado en lugar de clasificar limpiamente un único vector unificado de 704 logits.

Observamos también en el torneo, que **el agente heurístico de capturas** (HeuristicAgent, 1129.4 Elo) **obtiene puntuaciones más altas de lo inicialmente esperado**. Esto se debe sobre todo a los datos con los que han sido entrenados los modelos: posiciones con capturas y situaciones tácticas han sido filtradas y hacen que los modelos hayan visto pocos jaques, capturas, etc, y por tanto tengan dificultades para responder ante estas, ya que se encuentran fuera de la distribución de entrenamiento. Además, en un tablero reducido de 5×5 , la proximidad constante de las piezas hace que cualquier despiste táctico de captura sea castigado al instante. A pesar de su simplicidad, el heurístico supera al baseline MLP y al modelo plano simple. Sin embargo, los modelos espaciales `spatial_nofact` (1168.8 Elo) y `spatial_fact` (1135.2 Elo) logran superar al heurístico. Esto demuestra que los agentes basados en Transformer espacial han aprendido conceptos posicionales más sutiles (estructura

de peones, control del centro y seguridad del rey) que les permiten maniobrar y tender trampas incluso sin búsqueda. Este efecto se mitiga al utilizar un modelo mayor, como se puede ver en el anexo A.3, y en los resultados de Elo asociados (Tabla 9).

Uno de los resultados más llamativos del torneo es el rendimiento de `simple_nofact`. En la evaluación supervisada estática de test, este modelo obtiene la mayor precisión de movimientos (Move Accuracy P@1 = 48.63 %) y de resultados (Result Accuracy = 71.88 %) de toda la categoría plana. No obstante, en juego real su rendimiento colapsa por completo, cayendo a 964.7 de Elo, situándose incluso por debajo del agente aleatorio (`RandomAgent`, 1000 Elo) y de la red MLP (`MLP_Baseline`, 1007 Elo).

Esto ilustra una cierta desconexión entre las métricas estáticas supervisadas y la capacidad de juego interactiva, explicable por factores como la acumulación de errores gradual, que puede llevar a posiciones *Out-of-Distribution* (OOD), o a la alta tasa de empates.

Un **detalle estadístico relevante** surge al analizar los enfrentamientos cruzados (Figura 8). El campeón `spatial_nofact` obtuvo una tasa de victoria del 46,5 % contra `HeuristicAgent` (es decir, el heurístico ganó el enfrentamiento directo por un estrecho margen). Sin embargo, `spatial_nofact` obtiene un Elo superior (1168.8) al de `HeuristicAgent` (1129.4). Esto se debe a la consistencia global: el modelo espacial es dominante contra los Transformers planos y el MLP (logrando por ejemplo un 75 % de victorias contra `simple_fact` y `simple_nofact`), mientras que el heurístico muestra un desempeño más irregular y menos dominante contra el resto de la tabla.

Cabe destacar, finalmente, que el ajedrez Gardner es un juego con tendencia al empate, dado que al ser un tablero tan pequeño situaciones de ahogado son muy comunes y el juego tiende a posiciones donde ambos jugadores se "traban", evitando dar ventajas al otro. Esto se puede ver claramente tanto en el torneo como en las estadísticas del dataset, donde la tasa de empates es significativamente alta.

7.1.6. Conclusión sobre la hipótesis 2

A partir de los resultados experimentales obtenidos, podemos concluir lo siguiente respecto a la **Hipótesis 2**:

- Se confirma parcialmente que la introducción de una representación espacial bidimensional (inductivo bias) mejora de manera estadísticamente significativa el aprendizaje del valor (+8.51 % de exactitud). Sin embargo, esta mejora espacial no se traduce en un incremento significativo en la precisión del movimiento supervisado (resultado de la cabeza de política).
 - Se refuta que el uso de un espacio de acciones factorizado (casillas origen y destino independientes) no solo no mejora el rendimiento, sino que perjudica significativamente tanto la precisión del movimiento como la del valor, introduciendo una complejidad redundante.
 - El mejor modelo obtenido supera en Elo a un modelo con conocimiento previo del valor de las piezas, aunque la tasa de victorias sea ligeramente inferior contra el mismo.
-

7.2. Efectos de la inicialización supervisada sobre la convergencia del aprendizaje por refuerzo

A continuación, se presentan y discuten los resultados obtenidos en el análisis del impacto de la inicialización supervisada frente a un entrenamiento desde cero, con el fin de verificar la hipótesis 3 5.1.1. De forma similar a los experimentos de ablación anteriores, se utilizaron 10 semillas independientes para las 2 variantes utilizadas (Tabula Rasa vs. Pre-entrenado), con un total de 300 iteraciones cada una, las cuales corresponden a pasos de recolección de trayectorias y entrenamiento de PPO. Del mismo modo que en los experimentos anteriores, los hiperparámetros utilizados se mantienen fijos y se detallan en el Anexo A.7. Junto a métricas intrínsecas del algoritmo de RL, como la recompensa media acumulada por iteración o la pérdida de la *surrogate loss* y entropía, se recogió una medida de Elo en enfrentamientos 1 a 1 contra 2 agentes base; un agente aleatorio, y un agente heurístico (ambos presentes en el experimento anterior).

Cabe destacar también, que el número de iteraciones de self-play utilizado es reducido para un problema como este (modelos como AlphaZero utilizan millones de iteraciones en entornos altamente optimizados), lo que sumado al tamaño relativamente pequeño del modelo utilizado, resulta en un modelo competente, pero cuya fuerza de juego real (contra un humano o un sistema como Stockfish) no es muy alta. Esto no es negativo, ya que la reducción explícita del problema es parte de este trabajo: el problema no es más que un medio para el fin de probar las hipótesis planteadas.

A continuación, el Algoritmo 2 detalla el procedimiento síncrono de recolección de trayectorias (*rollouts*) ejecutado en cada iteración de PPO sobre N entornos vectorizados en paralelo.

Algorithm 2 Recolección de Trayectorias (*Rollouts*) en PPO Paralelo

Require: Política π_θ , función de valor V_θ , N entornos vectorizados, horizonte T

Ensure: Búfer de trayectorias $\mathcal{D}$ para optimización Actor-Crítico

- 1: Inicializar estados de los entornos $\{s_0^{(i)}\}_{i=1}^N$ y contadores de repetición
- 2: **for** paso temporal $t = 0, 1, \dots, T - 1$ **do**
- 3: Convertir posiciones $\{s_t^{(i)}\}$ a tensores de características espaciales $x_t^{(i)}$
- 4: Evaluar red: logits $\mathbf{z}_t^{(i)}$ y valores $v_t^{(i)} \leftarrow V_\theta(x_t^{(i)})$ (sin cálculo de gradientes)
- 5: Consultar jugadas legales y crear máscaras binarias $M_t^{(i)} \in \{0, 1\}^{704}$
- 6: Enmascarar logits ilegales: $\tilde{\mathbf{z}}_t^{(i)} \leftarrow \text{where}(M_t^{(i)}, \mathbf{z}_t^{(i)}, -10^9)$
- 7: Calcular distribuciones de probabilidad: $p_t^{(i)} \leftarrow \text{softmax}(\tilde{\mathbf{z}}_t^{(i)})$
- 8: Muestrear acciones $a_t^{(i)} \sim p_t^{(i)}$ y calcular log-probabilidades $\log \pi_\theta(a_t^{(i)} | s_t^{(i)})$
- 9: Transicionar entornos en paralelo con $\{a_t^{(i)}\} \rightarrow$ recibir $\{s_{t+1}^{(i)}, r_t^{(i)}, d_t^{(i)}\}$
- 10: **for** cada entorno $i \in \{1, \dots, N\}$ **do**
- 11: **if** el episodio ha finalizado ($d_t^{(i)} = \text{verdadero}$) **then**
- 12: Reiniciar entorno $s_{t+1}^{(i)} \leftarrow s_{\text{inicio}}$
- 13: **if** el agente juega con negras **then**
- 14: Ejecutar movimiento inicial del oponente en $s_{t+1}^{(i)}$
- 15: **end if**
- 16: **end if**
- 17: **end for**
- 18: Almacenar la tupla $(x_t^{(i)}, a_t^{(i)}, \log \pi_\theta, v_t^{(i)}, r_t^{(i)}, d_t^{(i)}, M_t^{(i)})$ en el búfer $\mathcal{D}$
- 19: **end for**
- 20: **return** Búfer de trayectorias $\mathcal{D}$

7.2.1. Resultados del entrenamiento

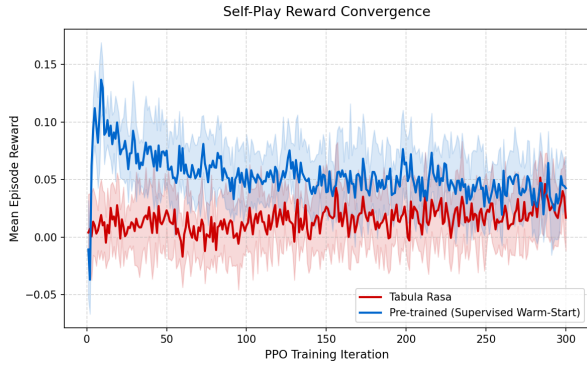
Las figuras 7 presentan información acerca de la evolución del entrenamiento. Cabe destacar una serie de puntos sobre estas. Observando la evolución de la recompensa media, se aprecia una tendencia: ambas recompensas, en lugar de crecer de forma sostenida, se estabilizan alrededor de cierto punto. Esto es algo normal para el caso de RL por self-play, ya que la política no se mantiene fija, sino que evoluciona y se adapta a la mejor versión de sí misma, haciendo que obtener recompensas sea más difícil a medida que el entrenamiento avanza.

La recompensa media acumulada del modelo preentrenado comienza alta en torno a 0.10, y se estabiliza en torno 0.04-0.05, indicando que el modelo encuentra gradualmente más difícil obtener recompensas (buena señal, ya que quiere decir que le resulta más difícil ganar a su versión anterior y que por lo tanto esta es más fuerte). Un resultado inverso se obtiene en el modelo entrenado desde cero, cuya recompensa media comienza entorno a 0, y aumenta hasta 0.3-0.4. De nuevo, esto es lógico, ya que este modelo inicia con conocimiento cero del juego, y por tanto no tiene una política adecuada para elegir acciones ni una cabeza de valor adecuada para estimar la ventaja.

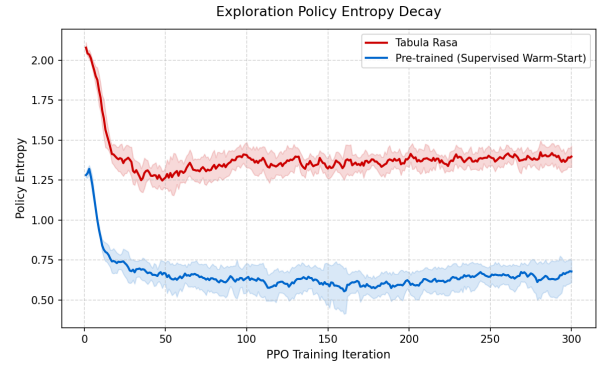
En cuanto a las pérdidas de entrenamiento, podemos observar que tanto la pérdida de entropía como la pérdida total se estabilizan tras una serie de iteraciones. Esto es buena señal a nivel de entrenamiento, aunque las pérdidas de PPO no son especialmente interpretables por sí mismas, dado que tienen varios componentes (clip loss, value loss, entropy loss) que se combinan

en una única métrica, y pueden tener valores negativos, por ejemplo en el caso de la política antigua siendo mayor que la política nueva ($\log(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)})$ donde $\pi_{\theta}(a_t|s_t) < \pi_{\theta_{old}}(a_t|s_t)$), además de que esta sigue dependiendo de un objetivo móvil y de una distribución de datos es cambiante.

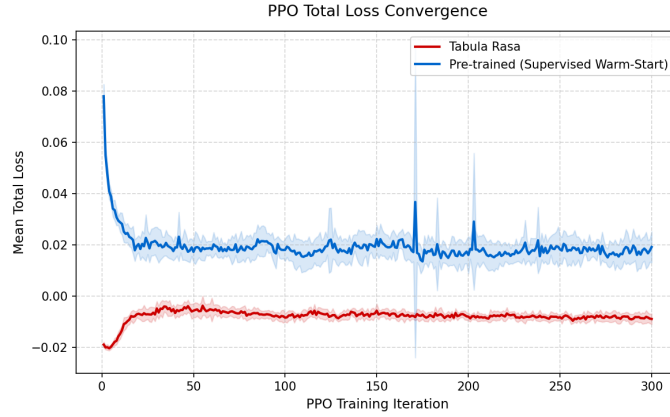
- TODO Apuntar a qué puede significar interpretativamente el escalón entre las pérdidas, resultados, recompensas... No es que ambos empiecen en el mismo sitio y uno .acelere” más, si no que ya para empezar, comienzan en niveles totalmente distintos Razonar sobre por qué el PPO es buena idea dado esto. Si tenemos un ”prior”, como por ejemplo la estimación de la ventaja dada por el crítico (value head), las trayectorias obtenidas serán más estables??



(a) Evolución de la recompensa media por episodio en self-play.



(b) Decaimiento de la entropía de la política durante el entrenamiento.



(c) Convergencia de la pérdida total media durante el entrenamiento PPO.

Figura 9: Curvas de aprendizaje de PPO para *tabula rasa* y pre-entrenado: recompensa media, entropía de política y pérdida total.

7.2.2. Evolución del Elo y razones de finalización de partida

Dado que la recompensa y las pérdidas no proporcionan una señal clara de la evolución o la capacidad de juego del modelo, se recurre a una serie de métricas extrínsecas adicionales, evaluando el Elo relativo contra dos agentes, uno aleatorio y otro heurístico, y registrando las razones de finalización de partida de entre las mencionadas en la sección 5.1.2. Las evaluaciones son aisladas, en torneos 1vs1 de 40 partidas, y se registraron cada 5 iteraciones de entrenamiento.

Las evaluaciones se realizan de forma aislada, por lo que no se muestra el Elo absoluto, sino las diferencias de Elo relativas entre las parejas de agentes. Los modelos entrenados jugaron exclusivamente contra si mismos, o con versiones anteriores de si mismos, por lo que esta evaluación es aislada y no está contaminada por posibles sesgos que el agente haya podido aprender a explotar en estos oponentes.

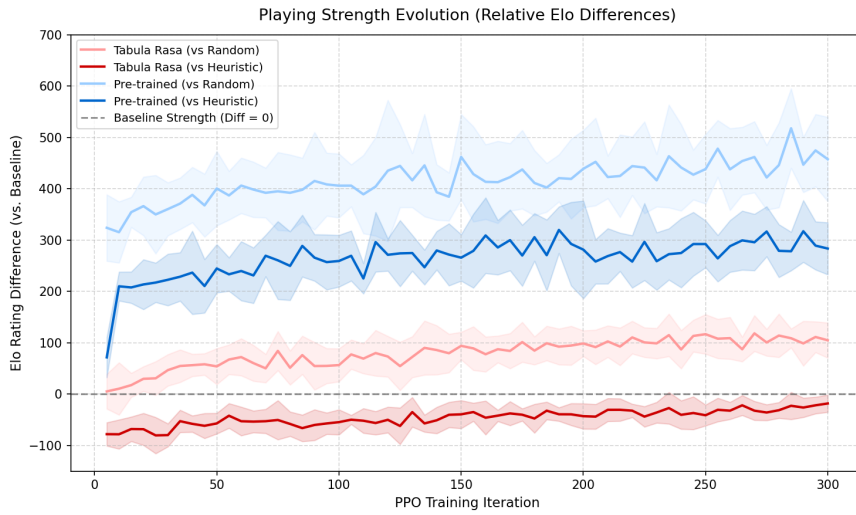
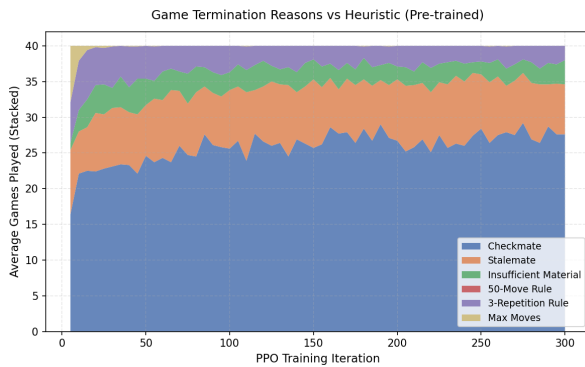
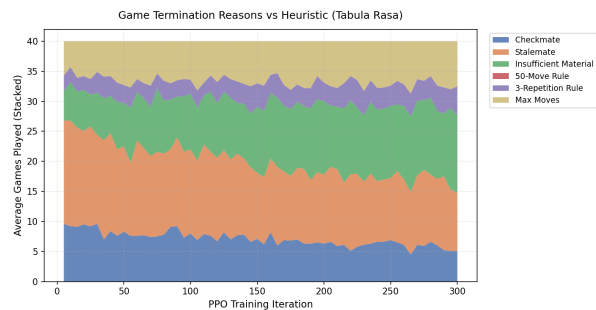


Figura 10: Evolución conjunta del Elo relativo contra agentes aleatorios y heurísticos.



(a) Modelo Pre-entrenado



(b) Modelo Tabula Rasa

Figura 11: Desenlaces de partida a lo largo del entrenamiento frente al baseline heurístico.

Se observa una tendencia clara en los resultados de elo relativo en la figura 10: el modelo preentrenado supera consistentemente al modelo *tabula rasa*, tanto frente al agente aleatorio

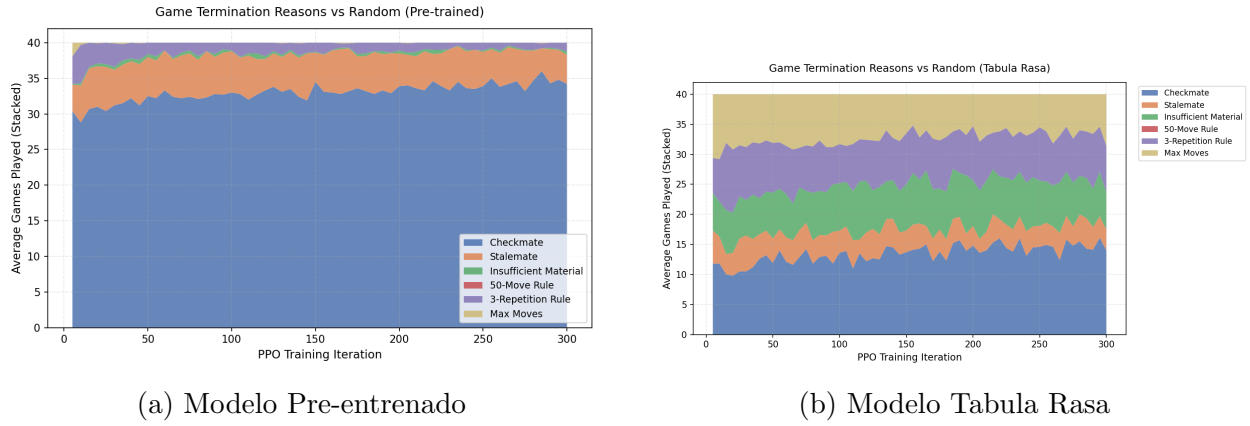


Figura 12: Desenlaces de partida a lo largo del entrenamiento frente al baseline aleatorio.

como frente al heurístico, y ambos obtienen un mayor Elo relativo frente a este último. Se aprecia también que en el número de iteraciones dadas, el modelo *tabula rasa* no logra alcanzar la capacidad de juego del agente heurístico, obteniendo un elo relativo negativo frente a este. Aunque queda fuera del alcance de este estudio, sería interesante evaluar el rendimiento de los modelos a lo largo de más iteraciones, en caso de que se de algún tipo de cambio en esta tendencia. Finalmente respecto a esta figura, dado que el Elo teórico sirve como un predictor del ratio de victorias^[23], se puede concluir que el modelo preentrenado es significativamente más fuerte que el modelo *tabula rasa*.

En cuanto a las razones de finalización de partida (ver figuras 11a y 12a), se observa que las partidas del modelo pre-entrenado aumentan gradualmente las victorias por jaque mate (contra ambos agentes), y se reducen razones de pérdida como material insuficiente o triple repetición. El rey ahogado se mantiene relativamente constante en todos los casos, ya que el tamaño del tablero y la naturaleza aleatoria de los agentes hacen probable estas situaciones.

Por otro lado el modelo *tabula rasa* muestra (11b, 12b) también mejoras graduales contra el agente aleatorio, con una ligera tendencia de aumento en mates y reducción en pérdidas por el máximo de movimientos, sin embargo en enfrentamientos contra el agente heurístico no logra mejorar su número de victorias por mate y empeora ligeramente sus resultados, aunque la tendencia de mejora en Elo indica que podría llegar a superarlo aumentando el número de iteraciones.

Es importante contextualizar estas observaciones. Recordemos que el modelo preentrenado ha visto pocas (o ninguna) posiciones tácticas como capturas / coronaciones / secuencias de mate, etc., y por ello no tiene conocimiento de estas situaciones, además de no tener visión de futuro explícita a través de la búsqueda; el aumento de mates en este contexto es una buena señal que indica que se modelan aspectos tácticos del ajedrez Gardner.

Por otro lado, los modelos parecen exhibir dificultad para reducir el número de finales por triple repetición y por número máximo de movimientos. Esto puede indicar problemas en la manera en la que el modelo procesa estos tokens de estado del juego, o que el entrenamiento no ha sido suficiente para que el modelo aprenda a evitar estas situaciones.

7.2.3. Evaluación estadística de la fuerza de juego final

Los resultados anteriores sirven como una medida de mejora del modelo a lo largo del entrenamiento. Junto a los resultados de la sección anterior, se observa un aumento tanto en la diferencia de ELO (que es interpretable como un aumento en la probabilidad de victoria) así como una mayor velocidad de convergencia para los agentes preentrenados.

Estos resultados se confirman a través de 2 tests estadísticos, uno paramétrico y otro no paramétrico (Welch's t-test y Wilcoxon rank-sum test sobre las 10 semillas utilizadas) 3. Ambos test confirman que la diferencia en Elo final es significativa ($p < 0,05$) entre los modelos preentrenados y los modelos con inicialización aleatoria.

Para analizar la velocidad de convergencia se define formalmente la iteración de estabilización como la primera iteración i donde el Elo medio iguala o supera el 90 % del Elo medio final siguiendo de acuerdo a la siguiente ecuación $I_{\text{estabilización}} = \min\{i \in \text{Iteraciones} \mid \text{Elo}_i \geq 0,9 \cdot \text{Elo}_{\text{final}}\}$

Observamos que el modelo pre-entrenado se estabiliza contra el heurístico en la iteración 70, mientras que el de tabula rasa necesita hasta la iteración 210 (convergencia $3,0\times$ más rápida) 3.

Métrica de Evaluación	Tabula Rasa	Pre-entrenado	t-stat	t-pval	W-stat	W-pval
Final Elo Diff vs. Random	$94,6 \pm 19,1$	$457,3 \pm 82,0$	12,52	$< 0,001$	3,06	0,0022
Iteración de Estabilización	105	90	—	—	—	—
Final Elo Diff vs. Heuristic	$-17,4 \pm 21,4$	$283,4 \pm 50,7$	15,06	$< 0,001$	3,06	0,0022
Iteración de Estabilización	210	70	—	—	—	—

Cadro 3: Comparación estadística detallada de la convergencia y fuerza final (Semilla 42).

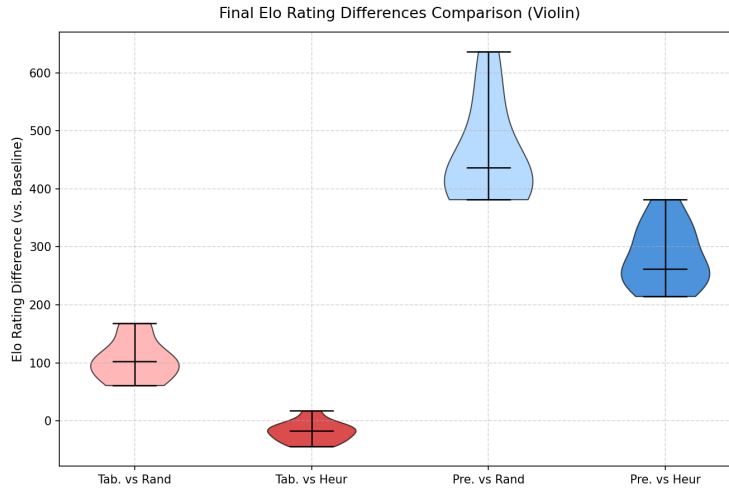


Figura 13: Gráfico de violín de la distribución de Elo final

7.2.4. Resultados del torneo mixto de 8 agentes

Las métricas de Elo obtenidas en las secciones anteriores se obtuvieron en enfrentamientos 1vs1 aislados, por lo que no indican una capacidad de victoria absoluta. Para co-calibrar

la capacidad y progresión de los distintos checkpoints, se realizó un torneo de round-robin similar al realizado en la hipótesis.

Se realizaron 100 partidas por pareja de agentes utilizando la semilla 42, una temperatura de ejecución de 0,225 y se ancló el agente aleatorio en 1000 Elo. La inferencia de los agentes en este torneo no se realizó como búsqueda de profundidad 1 como fue el caso del anterior torneo, sino que los agentes seleccionaron jugadas directamente a partir de la política escalada con la temperatura. El sistema de Elo se estimó del mismo modo al especificado en la sección 7.1.4.

Los agentes utilizados fueron: agentes aleatorio y heurístico como base, y 3 checkpoints de cada tipo de entrenamiento (tabula rasa y pre-entrenado), al 0 %, 50 % y 100 % de entrenamiento cada uno.

Nombre del Agente	Clasificación Elo (BT)
pretrained_it0	985,1
tabula_rasa_it0	996,1
RandomAgent	1000,0
tabula_rasa_it150	1080,2
HeuristicAgent	1092,6
tabula_rasa_it300	1147,7
pretrained_it150	1418,5
pretrained_it300	1424,4

Cadro 4: Resultados del torneo mixto de 8 agentes.

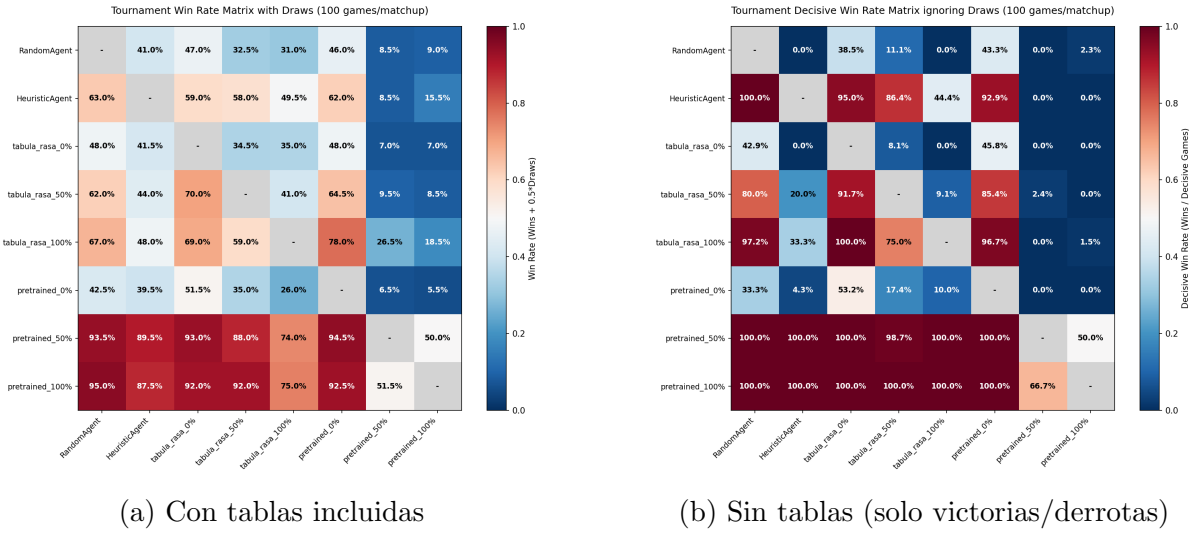


Figura 14: Matrices de tasa de victoria en el torneo de 8 agentes.

La interpretación de la Tabla 4 y las matrices de enfrentamiento cruzado (Figuras de tasa de victoria con y sin tablas) permite extraer conclusiones interesantes acerca de la hipótesis 3.

En primer lugar, se observa un fenómeno interesante en el rendimiento de **pretrained_it0** (985,1 Elo), que se sitúa por debajo del agente aleatorio de control. Dado que en este torneo se prescindió de la búsqueda de profundidad 1, y junto a la temperatura utilizas de 0,225, este resultado evidencia la fragilidad del modelo preentrenado, cuyos datos están pensados para búsqueda. Es probable que los datos fuera de distribución provoquen este comportamiento.

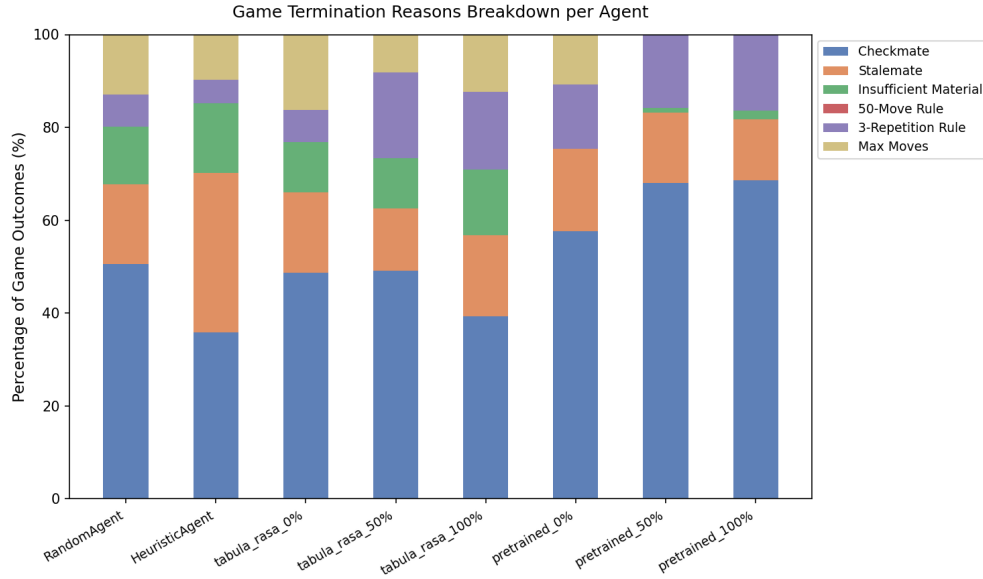


Figura 15: Gráfica apilada de los desenlaces de partida durante el torneo mixto.

En segundo lugar, a pesar de lo anterior, los resultados validan la efectividad del algoritmo PPO para el autoaprendizaje en el entorno de ajedrez 5×5 . Los modelos `tabula_rasa` muestran una progresión lineal y constante, a pesar de resultados negativos en el Elo relativo 7.2.2, ascendiendo desde los 996,1 Elo iniciales hasta 1147,7 Elo en la iteración 300, logrando superar con éxito al agente heurístico.

A pesar de esto, el modelo con inicialización supervisada mantiene una mejor capacidad de juego, confirmando las suposiciones de la hipótesis. El agente `pretrained_it150` alcanza los 1418,5 Elo, a pesar de que el aumento de puntuación da rendimientos decrecientes desde la iteración 150 a 300.

Por otro lado, la Figura 15 desglosa la distribución de los motivos de finalización de las partidas para cada agente a lo largo del torneo. Este análisis revela diferencias cualitativas drásticas en la estructura de las partidas:

En los agentes más débiles (`RandomAgent`, `HeuristicAgent` y los estados iniciales de `tabula_rasa`), una proporción muy significativa de los encuentros (entre el 10 % y el 15 %) finaliza por alcanzar el límite físico de movimientos (fijado en 100 halfmoves). Esto denota una dificultad para trazar planes de mate o coordinar piezas para obtener ventajas decisivas. En contraste, en los agentes preentrenados avanzados (`pretrained_50%` y `pretrained_100%`), las partidas por límite de movimientos desaparecen, lo que demuestra su mejora en su juego.

Los modelos `pretrained_50%` y `pretrained_100%` registran una subida en el número de jaques mates, siguiendo observaciones de datos anteriores 11a, del mismo modo que en el caso de las triples repeticiones.

Estos resultados no se pueden interpretar como completamente conclusivos por el bajo número de iteraciones y la selección dispersa de checkpoints del modelo (otros checkpoints podrían obtener mejores resultados), pero proporcionan una intuición de la mejor capacidad de los modelos preentrenados y la efectividad del PPO.

7.2.5. Conclusión sobre la hipótesis 3

- **Se confirma la hipótesis:** La inicialización supervisada proporciona un punto de partida óptimo que acelera sustancialmente el aprendizaje por refuerzo y permite alcanzar cotas de juego superiores en una fracción del tiempo de computación.
 - A pesar del número limitado de iteraciones, los modelos preentrenados muestran una mejora consistente en Elo y en la calidad de las partidas. Las condiciones del experimento no permiten descartar que entrenamientos más extensos podrían permitir a los modelos *tabula rasa* alcanzar o superar a los modelos preentrenados, pero la evidencia sugiere que la inicialización supervisada es un factor positivo para la convergencia y la fuerza de juego.
-

7.3. Resultados generales en el entorno de Minichess

Finalmente, la hipótesis 1 propuesta en la sección 5.1.1 busca justificar el uso del entorno reducido de ajedrez en el desarrollo de este trabajo.

Los resultados obtenidos en los distintos experimentos llevados a cabo permiten validar la hipótesis de que el Minichess es un entorno de estudio adecuado para la investigación en aprendizaje por refuerzo y redes neuronales.

Por un lado, su tamaño reducido permite una experimentación rápida y eficiente, eliminando la necesidad de recursos computacionales excesivos; todo el desarrollo de este proyecto se ha llevado a cabo con hardware de consumo, alternando entre computadoras (*Nvidia RTX 5060 Ti* 16GB + 32GB RAM, y *Nvidia RTX 3090* 24GB + 128GB RAM), y por tanto puede ser reproducido de forma sencilla y accesible.

Sin embargo, esta reducción del problema no implica una simplificación excesiva, ya que el Minichess mantiene suficiente complejidad para ser un entorno de estudio válido: a nivel de aprendizaje por refuerzo, esto se observa en la diferencia de elo y métricas obtenidas en la sección anterior 7.2; se puede realizar un entrenamiento en RL de forma ágil y con resultados positivos en este entorno.

En cuanto al uso de redes neuronales, se observa también una mejora consistente en los modelos transformer entrenados, a pesar de su tamaño reducido 6. Esta mejora cual está por encima de modelos más simples como MLP's en el caso del preentrenamiento supervisado 7, lo que justifica su uso, y permite obtener conclusiones sobre eficiencia de representación, arquitecturas y métodos de entrenamiento.

Adicionalmente, y como se menciona en los antecedentes 3, la condición *weakly-solved* de este juego podría dar pie en estudios futuros a evaluaciones más exigentes, contra los agentes teóricamente perfectos; algo posible en esta variante pero no en ajedrez estándar.

8. Conclusiones

En conclusión, las secciones 7.1, 7.2 y 7.3 presentan las aportaciones principales del trabajo, y los resultados obtenidos para las hipótesis planteadas. Los resultados obtenidos permiten confirmar que un sesgo inductivo bidimensional en la codificación del input es altamente

beneficioso para un modelo transformer aplicado al ajedrez en cuanto a la mejora de la predicción de valor de una posición, no tanto en el caso de la predicción de la política, y que la adición de una cabeza factorizada con la intención de re-orientar las predicciones a un formato de origen-destino no resulta beneficioso. Por otro lado, queda demostrado que el *warm-start* supervisado es significativamente beneficioso para PPO, acelerando el aprendizaje y permitiendo obtener mejores resultados en un menor tiempo de computación. Finalmente y a pesar de sus desventajas como la alta proporción de tablas, la falta de bases de datos de posiciones o la ausencia de ciertos movimientos, el entorno de Minichess ha demostrado ser un entorno de estudio adecuado para la investigación en aprendizaje por refuerzo adversarial y redes neuronales bajo restricciones de cómputo.

Dicho esto, me gustaría remarcar ciertas conclusiones a un nivel más personal. El objetivo de este trabajo era múltiple. Por un lado, este tiene una motivación personal; servir como una base simple y estable para comprender el preentrenamiento de modelos transformer y su posterior refinamiento mediante métodos de aprendizaje por refuerzo. Esto proporciona una intuición muy cercana tanto del funcionamiento de sistemas que usan esta estructura de entrenamiento (LLMs y modelos como AlphaGo), como de los retos que presentan.

Por otro lado, busca ser una base para explorar el trabajo investigativo: reproducir y expandir ideas de investigación ya existentes, como algunas de las presentadas en la sección de antecedentes. El aprendizaje por refuerzo es un campo complejo y lleno de retos; interpretabilidad, definición de funciones de recompensa, políticas cambiantes, inestabilidad de entrenamiento, baja eficiencia de muestreo, alto coste computacional y cuellos de botella en simulación, entre otros. Poder estudiarlo desde la perspectiva simplificada que ofrece el ajedrez Gardner permite mitigar algunos de estos problemas, y como ya se ha observado, aún así, obtener resultados interesantes y conclusiones relevantes a partir de una serie de hipótesis simples pero concretas.

Al plantear varias de estas hipótesis, yo mismo llegué a pensar que eran demasiado simples; algo que se “intuía” sin necesidad de demostración rigurosa.

Pero llegados al final de este trabajo, me doy cuenta de que esto no es así, y esto es precisamente lo que hace la investigación necesaria. Veamos por ejemplo, el caso de la hipótesis 3. Cualquiera que lleve un tiempo entrenando redes neuronales conoce de sobra la idea del *fine-tuning* sobre redes preentrenadas, una modalidad de *transfer-learning* ayuda a acelerar el aprendizaje sobre dominios concretos: una red se entrena con una gran cantidad de datos generales de cierto ámbito, y a continuación se especializa en otros específicos, omitiendo en este segundo entrenamiento el aprendizaje de muchas de las características comunes a ambos problemas.

Sin embargo, esta intuición común no funciona igual en todos los casos: en el caso de este trabajo, por ejemplo, una observación importante muestra un cambio en el modelo, dado que la tarea para la que se ha entrenado cambia de la fase de preentrenamiento a la fase de fine-tuning por RL. En la primera fase, la tarea objetivo consiste en predecir tanto el valor la política del estado, dados los datos obtenidos del motor de **fairy-stockfish**. Estos datos, que se obtienen de una búsqueda de profundidad 4, deben ser modelados por una red sin capacidad de búsqueda. Los resultados que observamos para esta fase 6 muestran como los resultados de la cabeza de valor son mejores, y estos son los que finalmente guían la elección de movimientos en las evaluaciones supervisadas: en los torneos, los agentes no eligen directamente con la política, sino que realizan un filtrado con esta y posterior elección

de la mejor jugada según la cabeza de valor.

Sin embargo, tras el RL, durante el cual el modelo se ve forzado a utilizar la cabeza de política como selector de movimientos, y la de valor como el estimador principal de la ventaja esperada, la tarea cambia ligeramente. Así, al ejecutar los torneos del mismo modo a los anteriores, la red toma acciones inesperadamente mal: no es hasta que se cambia el método a una selección directa por la política, como se entrenó en el RL, que no se obtienen buenos resultados. Esto es algo ligeramente contraintuitivo y que se entiende mejor dada la investigación realizada: los datos iniciales estaban pensados para búsqueda y funcionaban mejor aplicando una versión reducida de esta, pero la búsqueda no es compatible con PPO, dado que esto haría la pérdida de la política no diferenciable. Y aún así, esta red entrenada con PPO, que solo elige la siguiente jugada, vence a la versión preentrenada y a todas las demás que se probaron. Esto recuerda a la frase de Capablanca; cuando un reportero le preguntó cuántas jugadas en adelante calculaba en una posición, el respondió: “*Solo una, ¡la correcta!*”.

Otro caso curioso es el de la factorización de la política; al proponer la hipótesis, estaba bastante convencido de que esto ayudaría, aunque fuera ligeramente, al modelo a mejorar. Al fin y al cabo, tal y como había leído en papers como *Mastering chess with a transformer model*^[8], el uso de varias cabezas para proporcionar mayor señal al modelo era algo útil y común. Sin embargo, resultó no ser beneficioso en este caso, contrario a la intuición inicial (aunque el estudio de la causa exacta queda fuera del alcance de este trabajo).

Estos son solo un par de ejemplos de muchos que demuestran que hasta una idea aparentemente intuitiva merece cierta investigación. Finalmente, tras todo el trabajo y conclusiones, lo que más me llevo son preguntas. Esto no es algo malo, al contrario, es el resultado de haber profundizado en el tema y haber ganado una perspectiva más crítica sobre el mismo, y surge de la maravillosa curiosidad que me producen los temas cubiertos. En la sección siguiente, se presentan algunas de estas dudas y posibles líneas de investigación más prometedoras, en caso de continuar este trabajo a futuro.

9. Vías de trabajo futuro

Como se menciona anteriormente, para finalizar se presentan una serie de ideas, métodos y extensiones de este trabajo, que podrían resultar interesantes contando con más tiempo, y que podrían o bien mejorar los resultados o llevarlos en diferentes direcciones.

A un nivel más práctico, en cuanto a la arquitectura de la red neuronal y representaciones de la misma, existen numerosas mejoras que se podrían probar con el fin de mejorar el rendimiento:

- Por un lado, sería interesante testear el efecto de distintas codificaciones posicionales. La utilizada en la arquitectura final es una codificación absoluta, pero una codificación relativa, el uso de RoPE, podrían ser más adecuados para esta red. Ciertos estudios recientes están cuestionando la efectividad, o directamente la necesidad, de las codificaciones posicionales en transformers, por lo que también sería interesante probar el efecto de no utilizar ninguna.
- Sería interesante, visto el resultado negativo de añadir una cabeza de política extra, observar el efecto de una cabeza de valor adicional en el preentrenamiento. Experimentos

iniciales realizados con MLPs sugerían que el uso de una cabeza de valor orientada a la clasificación en lugar de a la regresión ($\{-1, 0, 1\}$ en lugar de $[-1, 1]$) proporcionaba resultados mejores, pero se acabó optando por la segunda dado que la asignación de valores a posiciones en inferecia es más sencilla.

- Con más tiempo y cómputo, una idea interesante habría sido asignar pesos a los componentes de la pérdida, dado que valor y política tienen magnitudes distintas. Estos pesos podrían ajustarse con un parámetro aprendible, o podrían ajustarse a base de búsqueda de hiperparámetros.
- Muchos trabajos previos de ajedrez invierten el tablero al cambiar el turno de juego del agente. En este caso no se realizó por una cuestión de simplicidad, pero esto podría mejorar el aprendizaje al no obligar a la red a memorizar patrones simétricos para cada color, (aunque la inclusión de un parámetro de "jugador activo" debería tener un efecto similar).
- Dado que el objetivo era utilizar agentes sin búsqueda, un dataset inicial sin filtros de búsqueda podría ser mas adecuado que el utilizado, pero los datasets de ajedrez Gardner no son algo común y esto probablemente requeriría modificar **fairy-stockfish**.
-

Algunas ideas interesantes se podrían aplicar también en cuanto a las dinámicas de aprendizaje del modelo en las fases tanto de aprendizaje supervisado como de self-play:

- Una idea interesante aplicable tanto al preentrenamiento como al aprendizaje por refuerzo es el uso de *curriculum learning*. Este paradigma de aprendizaje consiste en crear un currículum de complejidad progresiva, con la idea de que el modelo se entrene primero contra los datos / tareas más sencillas, pasando gradualmente a otras más difíciles. Esto estabiliza y acelera el entrenamiento. En el caso del preentrenamiento supervisado, esto podría verse, por ejemplo, como entrenar el modelo con conjuntos de datos obtenidos por búsqueda a profundidades incrementales; una serie de epochs sobre datos de búsqueda a profundidad 1, seguidas de datos a profundidad 2, etc. También podría darse como entrenamiento inicial sobre aperturas y finales, omitiendo el medio juego, o sobre puzzles (en caso de disponer de dichos datos).
- En cuanto a la fase de RL, algo similar se podría lograr con funciones de recompensa variables. Imaginemos, por ejemplo, una recompensa inicial de 2 partes; $R_{main} + c^T R_{boost}$ donde $0 \geq c < 1$ y T representa el rollout actual. Podríamos definir que R_{boost} devuelva recompensas al agente en función de distintos factores; las piezas capturadas, control de ciertas zonas del tablero, ataques al rey rival... Esto fomentaría ciertos comportamientos inicialmente en el agente, pero esta señal se disiparía a medida que T aumenta, forzándolo eventualmente volver a la recompensa inicial cuando llegue a 0.

Existen también una gran cantidad de mejoras posibles en cuanto a optimización y paralelización del entorno de RL, con el objetivo de reducir el cuello de botella en CPU:

- De cara a final del trabajo, descubrí una librería llamada PGX que implementa entornos de ajedrez 5x5 nativos en GPU mediante JAX. Esto permitiría ejecutar millones de pasos por segundo y escalar el experimento a redes más grandes y con más datos, al poder explotar el paralelismo de GPU para obtener los rollouts y no depender del límite de threads de la CPU. La migración no fue posible, ya que requeriría modificar

el input de la red neuronal, su arquitectura, y numerosos otros aspectos de código que ya estaban establecidos, pero su uso sería muy interesante.

- También existe la posibilidad, en caso de disponer de numerosos computadores con GPU's, de utilizar herramientas de computación distribuida como MPI, para ejecutar entornos en distintos dispositivos y reunir las transiciones resultantes a través de red.

Finalmente, me gustaría dejar algunas ideas adicionales que podrían llevar la investigación realizada en este trabajo en nuevas direcciones, o profundizar en la misma:

- La visualización de mapas de atención del Transformer y el estudio de circuitos internos^[24] podría ilustrar si distintas cabezas se especializan en tareas concretas, como el control de líneas, la defensa del rey o la detección de amenazas, y mostrar posibles redundancias o fallas en la red.
- Resultaría interesante investigar si al entrenar con datos de mayor profundidad (profundidad 2, profundidad 3, profundidad 4...) el modelo aprende a "simular una búsqueda interna (similar al trabajo de Jenner et al.^[10]). ¿Es posible que aprender la distribución de datos a mayores profundidades fuerze al modelo a desarrollar algún mecanismo interno de búsqueda?
- Se conoce que el escalado conjunto de datos, parámetros y cómputo mejora el rendimiento de los transformers^[25]. Resultaría de gran interés comprobar si la capacidad de juego directa (*one-shot*, sin búsqueda explícita) sigue leyes de escalado predecibles al aumentar el tamaño del modelo y la profundidad (complejidad) de los datos, en lugar de la cantidad de los mismos.
- Este trabajo se ha realizado intencionalmente sin proporcionar a los agentes mecanismos de búsqueda en el espacio de estados del ajedrez, pero podría resultar interesante observar el impacto que esta puede tener sobre el Elo base de un agente. ¿Qué mejora relativa de Elo introducen distintas profundidades de búsqueda sobre una misma red evaluadora de posiciones? ¿Hasta que punto es atribuible la fuerza de los agentes modernos a la búsqueda VS. al conocimiento interno de sus redes neuronales?
- Resultaría interesante evaluar la capacidad de la red para transferir conocimiento y adaptarse a tableros de mayor tamaño (8×8) o a variantes con posiciones iniciales modificadas, como las distintas variantes del ajedrez 5x5^[26].
- Finalmente, resultaría interesante indagar la causa de la falta de mejora en la política durante el preentrenamiento con representación espacial, así como el efecto negativo de la factorización de la política. Esto podría indicar que el modelo no es capaz de aprender a generalizar correctamente la política, o que la arquitectura no es adecuada para ello.

Referencias

- [1] M. R. Segal, “Chess, Chance and Conspiracy,” *Statistical Science*, vol. 22, n.º 1, feb. de 2007, ISSN: 0883-4237. DOI: 10.1214/088342306000000574. URL: <http://dx.doi.org/10.1214/088342306000000574>.
- [2] R. S. Sutton. “The Bitter Lesson.” (2019), URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [3] D. Silver, A. Huang, C. J. Maddison et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, n.º 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961. URL: <https://www.nature.com/articles/nature16961>.
- [4] D. Silver, T. Hubert, J. Schrittwieser et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, n.º 6419, pp. 1140–1144, 2018. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404>.
- [5] J. Jumper, R. Evans, A. Pritzel et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, n.º 7873, pp. 583–589, 2021. URL: <https://www.nature.com/articles/s41586-021-03819-2>.
- [6] Y. Nasu, “Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi,” en *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 1–7. URL: <https://github.com/asdfjkl/nnue>.
- [7] A. Ruoss, G. Delétang, S. Medapati et al., *Amortized Planning with Large-Scale Transformers: A Case Study on Chess*, 2024. arXiv: 2402.04494 [cs.LG]. URL: <https://arxiv.org/abs/2402.04494>.
- [8] D. Monroe e P. A. Chalmers, *Mastering Chess with a Transformer Model*, 2024. arXiv: 2409.12272 [cs.LG]. URL: <https://arxiv.org/abs/2409.12272>.
- [9] D. Monroe, G. Eilender, P. Chalmers, Z. Tang e A. Anderson, *Chessformer: A Unified Architecture for Chess Modeling*, 2026. arXiv: 2605.19091 [cs.LG]. URL: <https://arxiv.org/abs/2605.19091>.
- [10] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons e S. Russell, *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*, 2024. arXiv: 2406.00877 [cs.LG]. URL: <https://arxiv.org/abs/2406.00877>.
- [11] Daniel Monroe. “Transformer Progress in Leela Zero.” (2024), URL: <https://lczero.org/blog/2024/02/transformer-progress/>.
- [12] D. Pritchard e J. Beasley. “Encyclopedia of Chess Variants.” (2007), URL: <https://www.jsbeasley.co.uk/encyc/encyc.pdf>.
- [13] M. Mhalla e F. Prost, “Gardner’s Minichess Variant is solved,” *CoRR*, vol. abs/1307.7118, 2013. arXiv: 1307.7118. URL: <http://arxiv.org/abs/1307.7118>.
- [14] C. E. Shannon, “Programming a Computer for Playing Chess,” *Philosophical Magazine*, vol. 41, n.º 314, pp. 304–317, 1950.
- [15] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention is All you Need,” en *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio et al., eds., vol. 30, Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, “Proximal Policy Optimization Algorithms,” *CoRR*, vol. abs/1707.06347, 2017. arXiv: 1707.06347. URL: <http://arxiv.org/abs/1707.06347>.
- [17] L. Weng. “Reward Hacking in Reinforcement Learning.” (2024), URL: <https://lilianweng.github.io/posts/2024-11-28-reward-hacking/>.

- [18] R. Xiong, Y. Yang, D. He et al., “On Layer Normalization in the Transformer Architecture,” *CoRR*, vol. abs/2002.04745, 2020. arXiv: 2002.04745. URL: <https://arxiv.org/abs/2002.04745>.
- [19] A. Gupta, A. Ozdemir e G. Anumanchipalli, *Geometric Interpretation of Layer Normalization and a Comparative Analysis with RMSNorm*, 2025. arXiv: 2409.12951 [cs.LG]. URL: <https://arxiv.org/abs/2409.12951>.
- [20] Wikipedia contributors. “Bradley-Terry model - Wikipedia.” Consultado o 1 de xullo de 2026. (2026), URL: https://en.wikipedia.org/wiki/Bradley%E2%80%93Terry_model#Definition.
- [21] J. Czech, J. Blüml, K. Kersting e H. Steingrimsson, *Representation Matters for Mastering Chess: Improved Feature Representation in AlphaZero Outperforms Switching to Transformers*, 2024. arXiv: 2304.14918 [cs.AI]. URL: <https://arxiv.org/abs/2304.14918>.
- [22] B. Peng, T. Gigant e J. Quesnelle, *Efficient Pre-Training with Token Superposition*, 2026. arXiv: 2605.06546 [cs.CL]. URL: <https://arxiv.org/abs/2605.06546>.
- [23] ChessBase. “The Elo rating system - Correcting the expectancy tables.” (2011), URL: <https://en.chessbase.com/post/the-elo-rating-system-correcting-the-expectancy-tables>.
- [24] J. Kormu, *lc0-attention-visualizer*, 2022. URL: <https://github.com/jkormu/lc0-attention-visualizer>.
- [25] J. Hoffmann, S. Borgeaud, A. Mensch et al., *Training Compute-Optimal Large Language Models*, 2022. arXiv: 2203.15556 [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- [26] Wikipedia contributors. “Minichess - Wikipedia.” Consultado o 1 de xullo de 2026. (2026), URL: https://en.wikipedia.org/wiki/Minichess#Ajedrez_5x5.
- [27] V. Godbole, G. E. Dahl, J. Gilmer, Z. Nado e C. J. Shallue. “Deep Learning Tuning Playbook.” Google published this outside github too: <https://developers.google.com/machine-learning/guides/deep-learning-tuning-playbook>. (2023), URL: https://github.com/google-research/tuning_playbook.

A. Anexos

A.1. Hiperparámetros del entrenamiento y estudio de ablación

En esta sección se detallan los hiperparámetros fijos empleados en las fases de entrenamiento y, en particular, en el estudio de ablación de arquitectura diseñado para responder a la Hipótesis 2. Este estudio sistemático varía la representación del tablero (*simple* vs. *spatial*) y la utilización o no de la cabeza de política factorizada, repitiéndose bajo un conjunto de semillas aleatorias distintas para evaluar la estabilidad del aprendizaje y mitigar efectos del ruido estocástico.

Siguiendo las recomendaciones del *Deep Learning Tuning Playbook* de Google^[27], para realizar comparaciones rigurosas sobre las variables científicas del modelo se mantuvieron fijos los hiperparámetros. El uso de un tamaño de lote grande ($B = 16384$) maximiza el paralelismo y la estabilidad del gradiente en la GPU pero requiere un ajuste de los hiperparámetros del optimizador (tasa de aprendizaje y estabilidad numérica) obtenidos mediante un proceso de ajuste de Optuna.

Cabe destacar el tamaño de la dimensión de embedding en 64. Este es un valor relativamente bajo (modelos como GPT-2 tienen una dimensión de embedding de 768), pero es adecuado dadas las limitaciones computacionales. Testing informal demostró que aumentar la dimensión de embedding aporta mejoras significativas, junto con el aumento del número de datos de entrenamiento (siguiendo la intuición de las leyes de escalado). El aumento del número de bloques transformer también mejora el rendimiento, pero todo esto a costa de un mayor tiempo de entrenamiento.

La Tabla 5 detalla la configuración y valores específicos de estos hiperparámetros fijos empleados durante las 30 épocas del estudio de ablación.

Los hiperparámetros fijos se eligieron también siguiendo las recomendaciones del *tuning playbook* de DeepMind: modelos simples al inicio, learning rate constante para evitar complicaciones, maximizar el batch size para un entrenamiento más rápido, tunear solo los hiperparámetros importantes para el batch (parámetros de optimizador, learning rate). *"Our guiding principle is to always use simplest, lowest resource configuration that yields good results."*

El cambio de batch size requirió de ajustes varios, por lo mencionado en la guía: *"The hyperparameters that interact most strongly with the batch size, and therefore are most important to tune separately for each batch size, are the optimizer hyperparameters (e.g. learning rate, momentum) and the regularization hyperparameters."*

Cabe destacar que los resultados obtenidos se obtuvieron para un conjunto fijo de hiperparámetros. Esto no quita que puedan existir otros conjuntos de hiperparámetros que den mejores resultados, pero para el estudio de ablación realizado no se variaron los hiperparámetros más allá de los que se querían estudiar. No se asegura independencia condicional entre distintos conjuntos de hiperparámetros, y esto limita la extrapolación de resultados a otros conjuntos de hiperparámetros. *"Fixed hyperparameters will have their values fixed in the current round of experiments. These are hyperparameters whose values do not need to (or we do not want them to) change when comparing different values of the scientific hyperparameters. By fixing certain hyperparameters for a set of experiments, we must accept that conclusions derived from the experiments might not be valid for other settings of the fixed hyperparameters. In other words, fixed hyperparameters create caveats for any conclusions we draw from the experiments."* Referencia: Deep Learning Tuning Playbook, Google 2023

Hiperparámetro (Símbolo)	Valor	Descripción / Contexto Experimental
Dataset de entrada	<code>d4_val.txt</code>	Dataset filtrado obtenido a partir de partidas generadas a profundidad 4.
Split train/validation	97 % / 3 %	División del dataset en conjuntos de entrenamiento y validación.
Dimensión de embedding (d_k)	64	Dimensión de los embeddings de entrada y proyección interna.
Número de bloques (N)	3	Número de bloques encoder del Transformer en el backbone.
Épocas de entrenamiento	30	Pasadas completas sobre el dataset.
Tamaño de lote (B)	16384	Batch size grande para aprovechamiento de GPU.
Backend de atención	<code>math</code>	Kernels de SDPA optimizados para Tensor Cores.
Autocast de precisión	<code>none</code>	Sin autocast, entrenamiento en float32.
Precisión de matmul	<code>high</code>	Precisión para operaciones de multiplicación matricial con Tensor Cores.
Tasa de aprendizaje (lr)	0.002842	Tasa de aprendizaje optimizada tras la exploración de hiperparámetros con Optuna.
Weight decay (λ)	2×10^{-5}	Decaimiento de pesos (penalización L2) para AdamW.
AdamW β_1	0.9244	Coefficiente de promedio móvil del gradiente de primer orden en AdamW.
AdamW β_2	0.999	Coefficiente de promedio móvil del gradiente de segundo orden (por defecto).
AdamW ϵ	$3,6618 \times 10^{-9}$	Parámetro de estabilidad numérica para evitar divisiones por cero.
Semillas aleatorias ($Seed$)	912, 32, 5000, 1, 4474, 42, 123, 2026, 777, 8765	Múltiples semillas para evaluar estabilidad estocástica de los resultados.

Cadro 5: Hiperparámetros fijos empleados en las ejecuciones del estudio de ablación de arquitectura.

A.2. Estadísticas y análisis del conjunto de datos

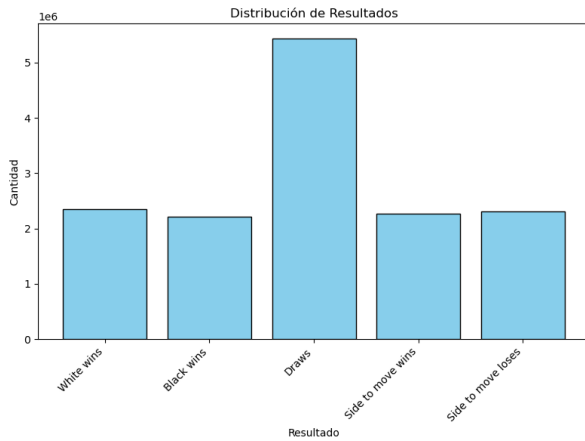
Para comprender mejor la composición del conjunto de datos de entrenamiento, se ha generado un reporte estadístico detallado sobre una muestra de 10,005,000 posiciones analizadas por **fairy-stockfish**. A continuación se muestran las distribuciones clave a través de varias gráficas representativas.

La Figura 16 muestra la distribución de resultados de las partidas de donde proceden las posiciones del conjunto de datos. Como se observa, el ratio de tablas es mayoritario, y parece que existe una ligera ventaja del blanco en el Gardner Minichess.

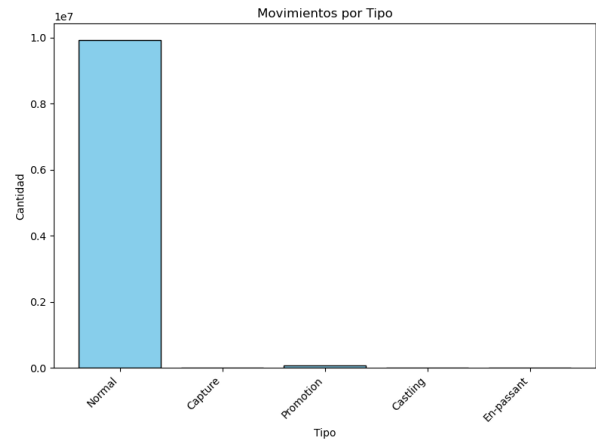
La Figura 17 ilustra la distribución de la diferencia de material evaluado por Fairy-Stockfish y la distribución del número de piezas totales en el tablero.

La Figura 18 muestra la distribución espacial de la frecuencia de aparición de los reyes blanco y negro sobre las casillas del tablero de 5×5 .

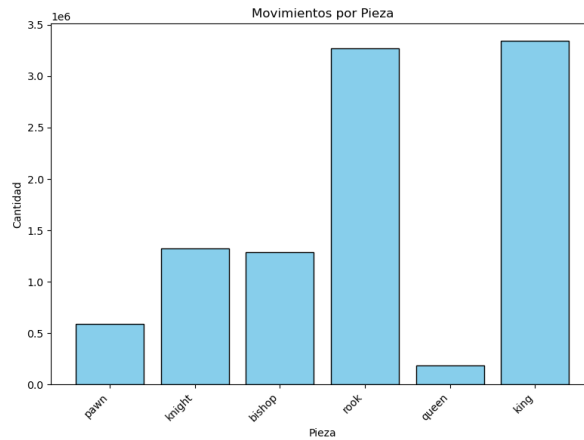
La Figura 19 muestra la distribución de la casilla de origen de los movimientos del jugador blanco y negro, y la figura 20 muestra lo mismo para la casilla de destino.



(a) Distribución de resultados.

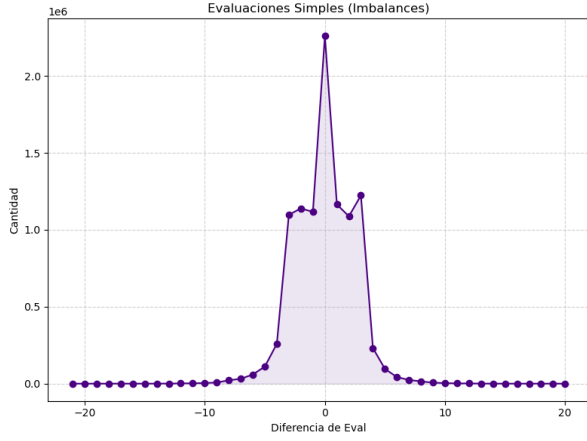


(b) Movimientos por tipo.

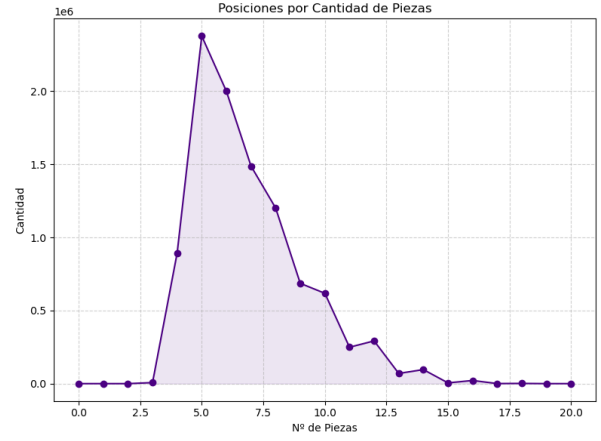


(c) Movimientos por piezas.

Figura 16: Distribución de resultados globales y tipo de movimientos.

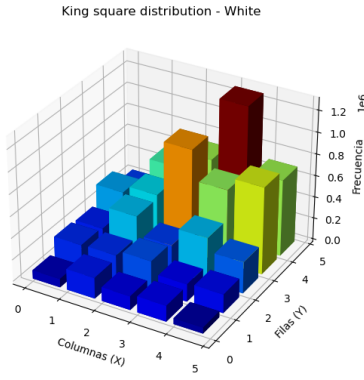


(a) Evaluaciones de desequilibrio (imbalances).

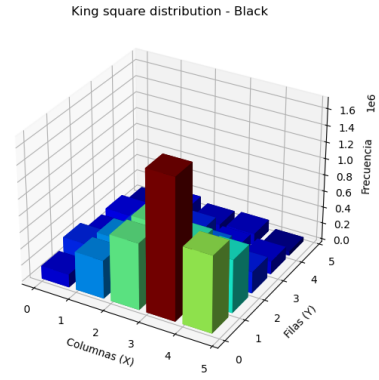


(b) Posiciones por número de piezas.

Figura 17: Distribución de balances materiales y densidad de piezas.

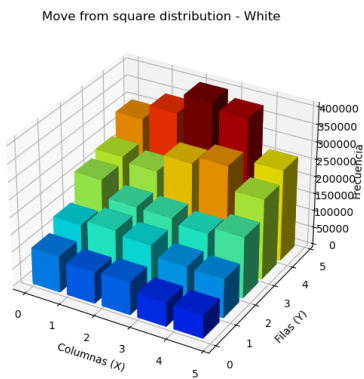


(a) Casillas del rey blanco.

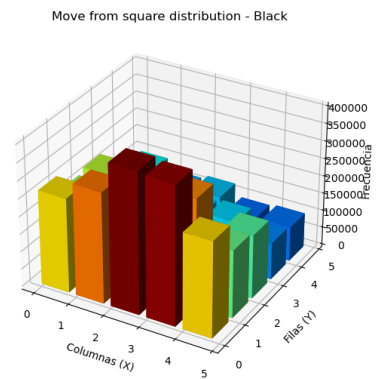


(b) Casillas del rey negro.

Figura 18: Distribución espacial de las posiciones de los reyes en el tablero.

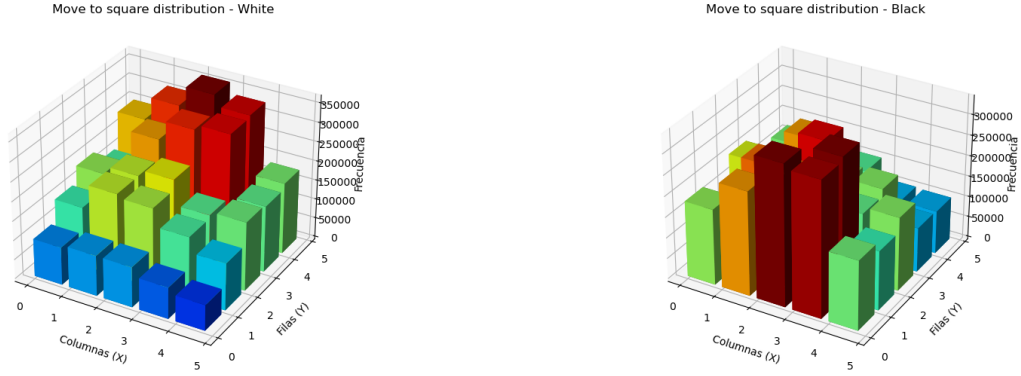


(a) Distribución de movimientos del jugador blanco: casilla de origen



(b) Distribución de movimientos del jugador negro: casilla de origen

Figura 19: Distribución de movimientos del jugador blanco y negro: casilla de origen.



(a) Distribución de movimientos del jugador blanco: casilla de destino (b) Distribución de movimientos del jugador negro: casilla de destino

Figura 20: Distribución de movimientos del jugador blanco y negro: casilla de destino.

A.3. Resultados adicionales del estudio de ablación sobre un modelo mayor

Para corroborar la validez general de las conclusiones obtenidas en el estudio de ablación y descartar que los efectos de la representación dependan de una capacidad de red muy limitada, se replicó la metodología de experimentación sobre un modelo Transformer de mayor tamaño y profundidad.

En concreto, se incrementó la dimensión de los embeddings (d_k) a 128 (frente a 64 en el modelo base) y el número de bloques encoder del Transformer a 5 (frente a 3 en la arquitectura de ablación base). Para asegurar la robustez estadística, este modelo mayor se entrenó de forma independiente bajo 8 semillas aleatorias diferentes durante 30 épocas.

En la Tabla 6 se resume la comparativa del número total de parámetros entrenables y tamaño en memoria (en FP32) de las tres arquitecturas principales utilizadas en este trabajo.

Modelo	Backbone / Configuración	N.º de Parámetros	Tamaño en Memoria
Transformer dk64_n3	$d_k = 64$, $N = 3$ bloques transf.	212,649	0.81 MB
Transformer dk128_n5	$d_k = 128$, $N = 5$ bloques transf.	1,132,137	4.32 MB
MLP_Baseline	4 capas lineales de 1024 neuronas	5,270,212	20.10 MB

Cadro 6: Comparación de complejidad, parámetros entrenables y huella de memoria de los modelos evaluados.

A pesar de que el modelo **Transformer dk128_n5** incrementa el número de parámetros en un factor de aproximadamente $5,3\times$ respecto al de ablación base, sigue siendo significativamente más compacto que el modelo **MLP_Baseline** (el cual posee casi 5.27 millones de parámetros). La arquitectura Transformer demuestra una eficiencia de parámetros muy superior al MLP, ofreciendo mayor capacidad con un consumo de recursos significativamente menor.

A.3.1. Hiperparámetros de la red de mayor capacidad

Los hiperparámetros de entrenamiento del modelo de mayor tamaño se optimizaron de forma adaptada a su capacidad, reduciendo ligeramente el tamaño del lote ($B = 12800$) y ajustando la tasa de aprendizaje ($lr = 0,001378$) para un entrenamiento eficiente en GPU. La Tabla 7 muestra la receta de hiperparámetros fijos empleada.

Hiperparámetro (Símbolo)	Valor	Descripción / Contexto Experimental
Dataset de entrada	<code>d4_val.txt</code>	Dataset filtrado obtenido a partir de partidas generadas a profundidad 4.
Split train/validation	97 % / 3 %	División del dataset en conjuntos de entrenamiento y validación.
Dimensión de embedding (d_k)	128	Dimensión de los embeddings de entrada y proyección interna.
Número de bloques (N)	5	Número de bloques encoder del Transformer en el backbone.
Épocas de entrenamiento	30	Pasadas completas sobre el dataset.
Tamaño de lote (B)	12800	Batch size grande adaptado al tamaño de la memoria de la GPU.
Backend de atención	<code>math</code>	Kernels de SDPA optimizados para Tensor Cores.
Autocast de precisión	<code>none</code>	Sin autocast, entrenamiento en float32.
Tasa de aprendizaje (lr)	0.0013783	Tasa de aprendizaje optimizada para la escala del modelo.
Weight decay (λ)	0.04996	Decaimiento de pesos ajustado para regularización del modelo grande.
AdamW β_1	0.9053	Coefficiente de promedio móvil del gradiente de primer orden en AdamW.
AdamW β_2	0.999	Coefficiente de promedio móvil del gradiente de segundo orden (por defecto).
AdamW ϵ	$2,265 \times 10^{-8}$	Parámetro de estabilidad numérica en AdamW.
Semillas aleatorias (<i>Seed</i>)	123, 2026, 32, 42, 5000, 777, 8765, 912	8 semillas aleatorias independientes para comprobar consistencia.

Cadro 7: Hiperparámetros empleados en el entrenamiento del modelo **Transformer dk128.n5**.

A.3.2. Evaluación en test holdout

Los modelos grandes se evaluaron sobre el mismo conjunto de test estático. La Tabla 8 reporta las medias y desviaciones estándar de las métricas de test consolidadas a lo largo de las 8 semillas independientes:

A.3.3. Análisis estadístico y test de Tukey HSD

Se aplicaron los mismos tests estadísticos a las métricas del modelo grande para contrastar formalmente los resultados:

Modelo / Representación	Move Acc (P@1)	Precision@3	Precision@5
<code>simple_nofact</code> (Plano, Simple)	53,33 % $\pm$ 0,06 %	87,21 % $\pm$ 0,05 %	96,51 % $\pm$ 0,02 %
<code>simple_fact</code> (Plano, Factorizado)	53,01 % $\pm$ 0,11 %	86,96 % $\pm$ 0,08 %	96,40 % $\pm$ 0,04 %
<code>spatial_nofact</code> (2D, Simple)	53,63 % $\pm$ 0,33 %	87,41 % $\pm$ 0,21 %	96,59 % $\pm$ 0,09 %
<code>spatial_fact</code> (2D, Factorizado)	53,84 % $\pm$ 0,26 %	87,48 % $\pm$ 0,17 %	96,63 % $\pm$ 0,07 %

(a) Métricas de la cabeza de política (movimientos) del modelo grande.

Modelo / Representación	Result Acc	Value MSE
<code>simple_nofact</code> (Plano, Simple)	74,93 % $\pm$ 0,09 %	0,2548 $\pm$ 0,0007
<code>simple_fact</code> (Plano, Factorizado)	75,08 % $\pm$ 0,15 %	0,2522 $\pm$ 0,0012
<code>spatial_nofact</code> (2D, Simple)	82,28 % $\pm$ 0,10 %	0,1396 $\pm$ 0,0004
<code>spatial_fact</code> (2D, Factorizado)	81,99 % $\pm$ 0,04 %	0,1415 $\pm$ 0,0004

(b) Métricas de la cabeza de valor (resultado) del modelo grande.

Cadro 8: Rendimiento medio y desviación estándar de las métricas en test para el modelo grande `Transformer dk128_n5` (8 semillas $\times$ 4 variantes).

Exactitud de movimiento (política): El test de Shapiro-Wilk sobre la exactitud de movimientos arroja un p-valor de 0,0455, indicando una desviación menor de la normalidad teórica. El análisis ANOVA de una vía arrojó un p-valor de $6,19 \times 10^{-7}$, ratificando diferencias altamente significativas. El test post-hoc de Tukey HSD revela lo siguiente:

- Al dotar al modelo de mayor capacidad ($d_k = 128$, $N = 5$), las arquitecturas con codificación espacial superan levemente en Move Accuracy a las planas. La diferencia es estadísticamente significativa entre `spatial_fact` y `simple_nofact` (+0,51 %, $p_{adj} = 0,0009$), así como entre `spatial_fact` y `simple_fact` (+0,83 %, $p_{adj} = 0,0$), corroborando el beneficio de la codificación espacial.
- La factorización del espacio de acciones degrada menos el rendimiento que en el modelo pequeño, pero la comparación entre `simple_fact` y `simple_nofact` sigue mostrando una penalización estadísticamente significativa de -0,32 % ($p_{adj} = 0,0481$).

Exactitud de Resultado (Valor): El test de Shapiro-Wilk confirma una desviación de la normalidad ($p \approx 0,0$) como en la versión de dimensión 64. Aún así, el ANOVA global indica una significatividad enorme ($p = 1,90 \times 10^{-43}$). El test post-hoc de Tukey HSD ratifica:

- La codificación espacial 2D del tablero aporta una mejora muy significativa y consistente en la predicción del resultado. `spatial_nofact` supera a `simple_nofact` en +7,35 % ($p_{adj} = 0,0$), y `spatial_fact` supera a `simple_fact` en +6,91 % ($p_{adj} = 0,0$). Asimismo, el error MSE se reduce casi a la mitad (de 0,2548 a 0,1396).
- La variante espacial simple `spatial_nofact` supera a su contraparte factorizada `spatial_fact` en +0,29 % ($p_{adj} = 0,0001$).

Los boxplots de la Figura 21 ilustran la dispersión en test para las 8 semillas, confirmando la superioridad de los modelos espaciales en la cabeza de valor.

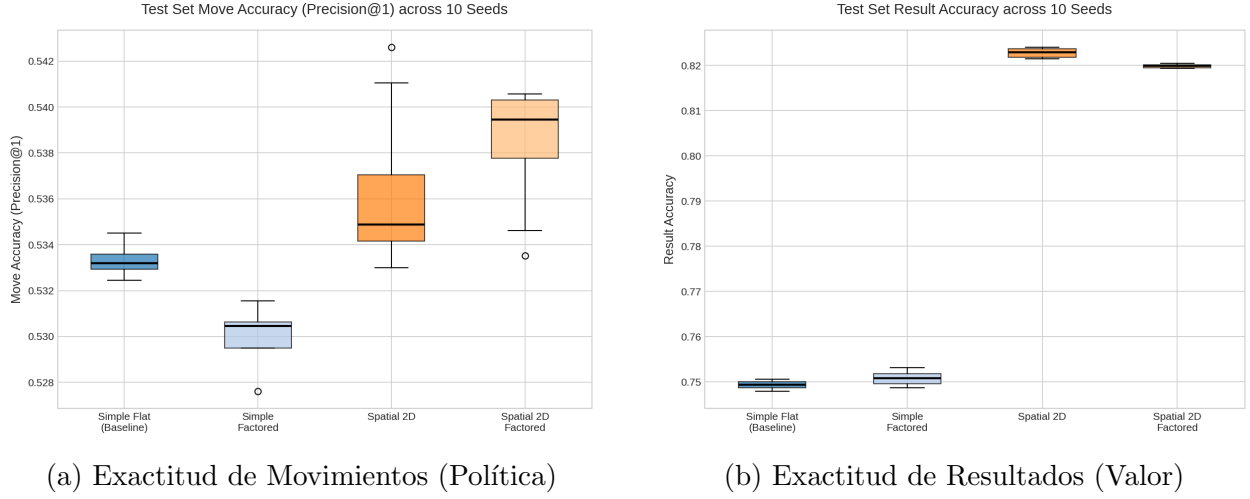


Figura 21: Distribución y variabilidad de las métricas en test para el modelo grande **Transformer dk128_n5** sobre las 8 semillas.

A.3.4. Curvas de aprendizaje del modelo grande

Las curvas de aprendizaje promediadas a lo largo de las 8 semillas independientes se detallan en la Figura 22. De forma similar a los modelos pequeños, las variantes espaciales muestran una convergencia superior y más rápida en la cabeza de valor (Figura 22b).

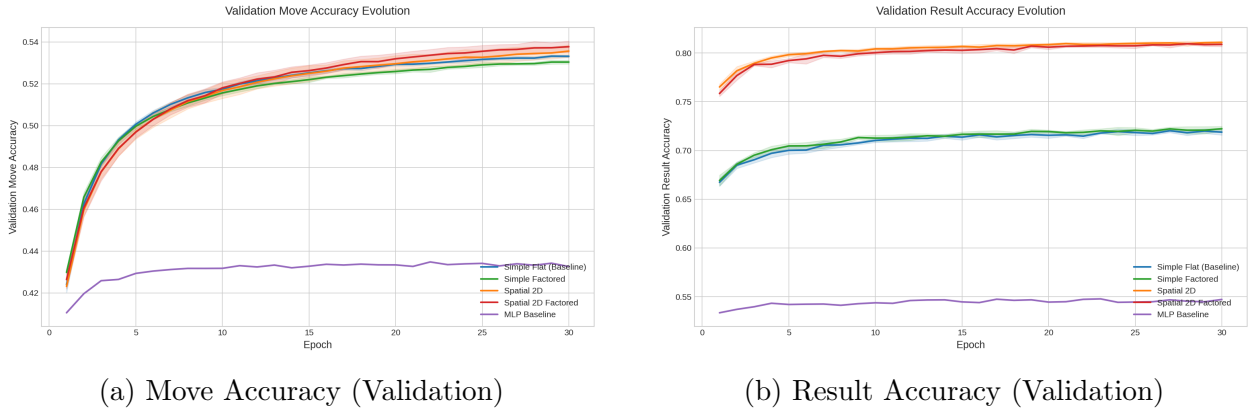


Figura 22: Curvas de aprendizaje promediadas para el modelo grande **Transformer dk128_n5** (con área sombreada de desviación estándar).

A.3.5. Torneo del modelo grande

Para complementar la evaluación y medir la habilidad práctica de juego de las variantes grandes, se ejecutó un torneo round-robin análogo al del modelo base. Los enfrentamientos se configuraron bajo las mismas condiciones (100 partidas por emparejamiento con intercambio de colores, límite de 100 movimientos, y temperatura de selección de jugada $\tau = 0,15$ para promover diversidad y mitigar loops repetitivos). Se utilizaron los checkpoints de los agentes entrenados bajo la semilla 42.

La Tabla 9 muestra los resultados de Elo obtenidos.

Agente	Arquitectura / Tipo	Elo Rating	Entropía Media Política
spatial_nofact	Transformer (2D, Simple)	1197.9	1.2778
simple_nofact	Transformer (Plano, Simple)	1173.8	1.3460
spatial_fact	Transformer (2D, Factorizado)	1159.1	1.2112
simple_fact	Transformer (Plano, Factorizado)	1141.3	1.2854
HeuristicAgent	Heurístico de capturas	1138.7	N/A
MLP_Baseline	Perceptrón Multicapa	1001.7	1.3063
RandomAgent	Selección aleatoria uniforme	1000.0	N/A

Cadro 9: Clasificación final y Elo de los agentes basados en el modelo grande Transformer dk128_n5 (Semilla 42).

A la vista de la clasificación de Elo, podemos extraer algunas conclusiones sobre el impacto de escalar la capacidad del modelo:

- A diferencia del modelo pequeño ($d_k = 64$), donde `simple_nofact` sufría un colapso en el torneo cayendo por debajo del agente aleatorio (964.7 Elo), al incrementar la capacidad del Transformer a $d_k = 128$, $N = 5$, la variante `simple_nofact` se recupera de manera notable, alcanzando el segundo puesto con **1173.8 Elo**.
- El modelo `spatial_nofact` se corona como el campeón del torneo con **1197.9 Elo**. La ventaja del sesgo inductivo espacial 2D se traduce en una superioridad posicional constante.
- Mientras que en el modelo pequeño el heurístico superaba al baseline plano, aquí las cuatro variantes Transformer superan de forma significativa al agente heurístico (1138.7 Elo), evidenciando que el incremento de capacidad permite al Transformer aprender patrones estratégicos profundos que superan la táctica codiciosa de capturas del heurístico. El elo máximo de estos modelos también supera al del modelo pequeño por +30 puntos.

Las matrices de tasas de victorias (con y sin empates) del torneo del modelo grande se ilustran en la Figura 23.

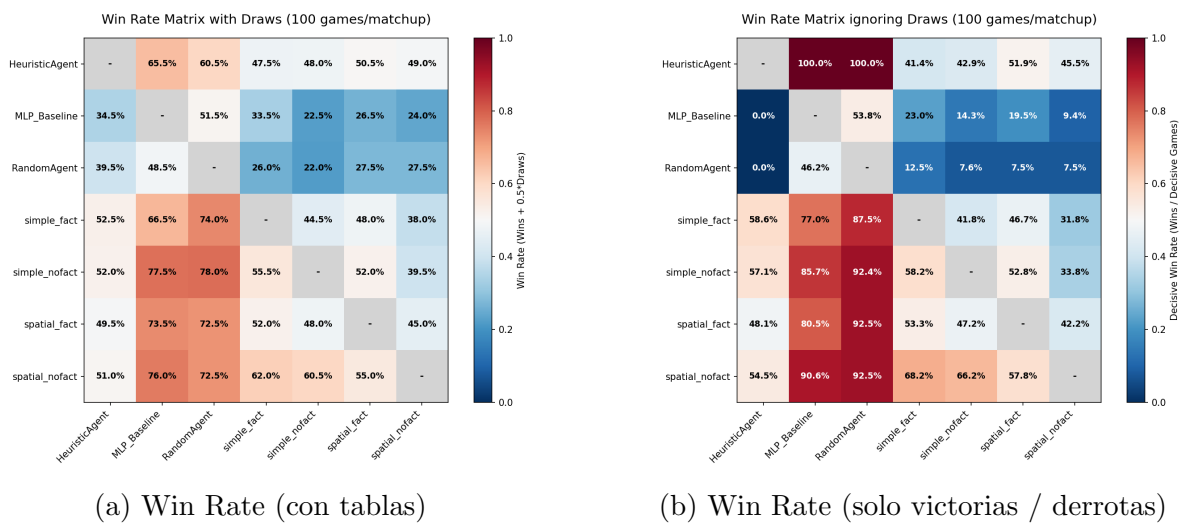


Figura 23: Matrices cruzadas de tasas de victorias para el torneo del modelo grande Transformer dk128_n5 (Semilla 42).

A.4. Formulación matemática de las funciones de pérdida

Las pérdidas individuales que componen la función de optimización global durante la fase de aprendizaje supervisado de los modelos se formulan matemáticamente de la siguiente manera:

1. **Pérdida de la cabeza de política principal ($\mathcal{L}_p$):** Se calcula mediante entropía cruzada multiclase clásica (*Cross-Entropy Loss*) aplicada sobre el vector de 704 logits de movimientos legales. Dado el movimiento objetivo $y \in \{1, \dots, 704\}$ (representado como un vector *one-hot* $\mathbf{y}$) y los logits predichos por el modelo $\hat{\mathbf{z}} \in \mathbb{R}^{704}$:

$$\mathcal{L}_p = - \sum_{c=1}^{704} y_c \ln \left(\frac{e^{\hat{z}_c}}{\sum_{k=1}^{704} e^{\hat{z}_k}} \right) \quad (3)$$

2. **Pérdida de la Cabeza de Valor ($\mathcal{L}_v$):** Se evalúa mediante el error cuadrático medio (*Mean Squared Error*, MSE) entre la predicción continua de valor $\hat{v} \in [-1, 1]$ (obtenida tras proyectar linealmente el token **CLS** y aplicar una activación $\tanh$) y el resultado objetivo de la partida $v \in \{-1, 0, 1\}$ (donde -1 indica derrota de las blancas, 0 indica tablas y 1 indica victoria de las blancas):

$$\mathcal{L}_v = (v - \hat{v})^2 \quad (4)$$

3. **Pérdida Auxiliar de la Política Factorizada ($\mathcal{L}_{\text{aux}}$):** Para los modelos factorizados, esta pérdida se desglosa en la suma de tres entropías cruzadas independientes aplicadas sobre las predicciones de la casilla de origen ($\mathcal{L}_{\text{from}}$), casilla de destino ($\mathcal{L}_{\text{to}}$) y el tipo de pieza de coronación ($\mathcal{L}_{\text{promo}}$):

$$\mathcal{L}_{\text{aux}} = \mathcal{L}_{\text{from}} + \mathcal{L}_{\text{to}} + \mathcal{L}_{\text{promo}} \quad (5)$$

donde cada término individual se formula empleando entropía cruzada convencional.

A.5. Evolución de las pérdidas por cabeza durante el entrenamiento

Para analizar cómo cada cabeza de salida aprende las características supervisadas en función de su arquitectura, la Figura 24 muestra la evolución de cada componente de pérdida en el conjunto de entrenamiento.

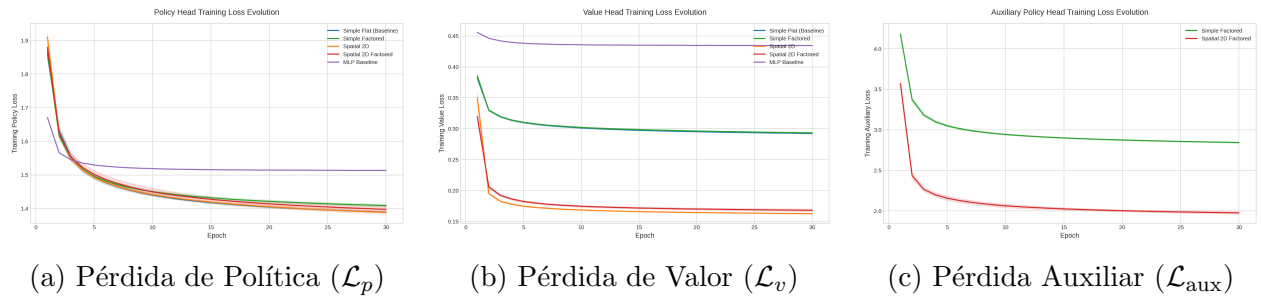


Figura 24: Evolución de las pérdidas de entrenamiento desglosadas por cabeza de salida en el estudio de ablación (media de 10 semillas $\pm$ desviación estándar).

Se puede observar claramente que:

- La pérdida de la cabeza de política ($\mathcal{L}_p$) converge a un valor similar para todos los modelos basados en Transformer, siendo notablemente menor que el baseline MLP. El uso de la cabeza auxiliar factorizada (`simple_fact` y `spatial_fact`) no altera sustancialmente la convergencia de la política principal.
 - La pérdida de la cabeza de valor ($\mathcal{L}_v$) muestra la superioridad de las codificaciones de entrada espaciales (`spatial_nofact` y `spatial_fact`), las cuales logran minimizar la pérdida a valores cercanos a 0.28, frente al 0.31 de los modelos planos.
 - La pérdida auxiliar ($\mathcal{L}_{aux}$) disminuye progresivamente durante el entrenamiento para las dos arquitecturas factorizadas, indicando que el modelo aprende a correlacionar y estructurar el origen-destino de los movimientos.
-

A.6. Resultados detallados de los tests de significatividad estadística

En esta sección se reportan de manera detallada las tablas y p-valores de los análisis estadísticos realizados sobre el estudio de ablación (Hipótesis 2) para las 10 semillas independientes, para la versión del modelo con 64 dimensiones (dk64).

A.6.1. Test de normalidad (Shapiro-Wilk)

Se evalúa la hipótesis nula de que los datos proceden de una distribución normal. Para la exactitud de movimientos (política), se obtiene; $p\text{-valor} = 0,127205$. Al ser $p > 0,05$, no se rechaza la hipótesis nula de normalidad, lo que fundamenta el uso de la prueba ANOVA paramétrica, que requiere que los datos sigan una distribución normal.

Para la exactitud del resultado (valor), el p-valor es prácticamente nulo ($p < 10^{-5}$). Esto indica una desviación de la normalidad teórica, por lo que no se puede usar ANOVA para comparar las medias de los grupos de valor. Aun así, dado el tamaño de la muestra y la magnitud de la diferencia estadística en los resultados que se observa en la figura 6b, y el hecho de que ANOVA es bastante robusto a las violaciones de normalidad bajo ciertas condiciones, y aún sin alcanzar las 30 muestras recomendadas para que el teorema central del límite sea efectivo, el test estadístico puede considerarse adecuado. Por simplicidad se mantiene ANOVA y Turkey para ambos casos, pero pruebas utilizando tests no paramétricos (Kruskal-Wallis y Mann-Whitney U) para el resultado de la cabeza de valor confirman la misma hipótesis obtenidas.

A.6.2. Test de diferencias globales (One-Way ANOVA)

El valor p devuelto por el test de ANOVA informa sobre la probabilidad de observar los datos observados, o unos más extremos, si la hipótesis nula (que todas las medias son iguales) fuera cierta. Por lo tanto, p-valores pequeños sugieren que es poco probable que las medias de los grupos sean iguales. Se evalúa si existen diferencias significativas entre las medias de los cuatro grupos comparados:

- **Exactitud de Movimiento (Política):** $p\text{-valor} = 6,92036 \times 10^{-10}$. Existe una diferencia altamente significativa entre los grupos de política.
- **Exactitud de Resultado (Valor):** $p\text{-valor} = 2,11741 \times 10^{-50}$. Diferencia enormemente significativa entre las representaciones de valor.

A.6.3. Comparaciones múltiples por parejas (Tukey HSD)

Se aplica el test de Tukey HSD con una tasa de error global controlada ($\text{FWER} = 0,05$). Las tablas de diferencias de medias (*meandiff*), p-valores ajustados (*p-adj*), intervalos de confianza y la decisión de rechazo se muestran en las Tablas 10 y 11. Recordemos que *meandiff* es la diferencia entre la media del primer grupo y la del segundo (grupo 2 - grupo 1), por lo que un valor positivo indica que el segundo grupo es mejor que el primero y viceversa. Por otro lado, H_0 sostiene que las medias de los grupos son iguales; si se rechaza quiere decir que existe una diferencia estadísticamente significativa entre las medias de los grupos, cuya "dirección" está indicada por el signo de *meandiff*. El intervalo de confianza informa sobre el rango de valores dentro del cual se encuentra la verdadera diferencia entre las medias de los grupos con un 95 % de confianza. Si el intervalo de confianza incluye el cero, no se puede rechazar la hipótesis nula. Finalmente, $p - adj$ es el p-valor ajustado; indica la probabilidad de rechazar la hipótesis nula.

Grupo 1	Grupo 2	Dif. Medias	p-adj	Int. Conf. 95 %	Rechazo H_0
simple_fact	simple_nofact	+0,0096	0,0000	[+0,0066, +0,0127]	Sí
simple_fact	spatial_fact	+0,0040	0,0055	[+0,0010, +0,0071]	Sí
simple_fact	spatial_nofact	+0,0082	0,0000	[+0,0052, +0,0112]	Sí
simple_nofact	spatial_fact	-0,0056	0,0001	[-0,0086, -0,0026]	Sí
simple_nofact	spatial_nofact	-0,0014	0,5926	[-0,0045, +0,0016]	No
spatial_fact	spatial_nofact	+0,0042	0,0036	[+0,0012, +0,0072]	Sí

Cadro 10: Comparación por parejas post-hoc de Tukey HSD para la exactitud del movimiento (Política).

Grupo 1	Grupo 2	Dif. Medias	p-adj	Int. Conf. 95 %	Rechazo H_0
simple_fact	simple_nofact	+0,0020	0,0664	[-0,0001, +0,0041]	No
simple_fact	spatial_fact	+0,0815	0,0000	[+0,0794, +0,0836]	Sí
simple_fact	spatial_nofact	+0,0871	0,0000	[+0,0850, +0,0892]	Sí
simple_nofact	spatial_fact	+0,0795	0,0000	[+0,0774, +0,0816]	Sí
simple_nofact	spatial_nofact	+0,0851	0,0000	[+0,0830, +0,0872]	Sí
spatial_fact	spatial_nofact	+0,0056	0,0000	[+0,0035, +0,0077]	Sí

Cadro 11: Comparación por parejas post-hoc de Tukey HSD para la exactitud del resultado (Valor).

A.7. Hiperparámetros PPO

En esta sección se detallan los hiperparámetros del algoritmo Proximal Policy Optimization (PPO) empleados durante los experimentos de autoaprendizaje (*self-play*) para la evaluación de la Hipótesis 3.

La Tabla 12 especifica la configuración del búfer de recolección de trayectorias, la estimación de ventaja (GAE), la función de pérdida multiobjetivo y la optimización.

Hiperparámetro (Símbolo)	Valor	Descripción / Contexto Experimental
Entornos paralelos (N)	100	Número de instancias concurrentes del entorno de ajedrez Gardner.
Pasos de rollout (T)	100	Pasos temporales ejecutados por entorno en cada iteración de recolección.
Transiciones por iteración	10,000	Volumen total de muestras ($N \times T$) recolectadas por iteración PPO.
Épocas de optimización (K)	8	Pasadas de optimización sobre las muestras recopiladas por iteración.
Tamaño de minibatch (B)	1024	Tamaño de lote empleado en cada paso de actualización del optimizador.
Tasa de aprendizaje (α)	2×10^{-4}	Tasa de aprendizaje fijada para el optimizador AdamW.
Optimizador	AdamW	Algoritmo de optimización por descenso de gradiente.
Factor de descuento (γ)	0.99	Factor de descuento para el cálculo del retorno acumulado futuro.
Parámetro GAE (λ_{GAE})	0.75	Coefficiente de sintonización sesgo-varianza en la estimación de ventaja GAE.
Coefficiente de recorte (ϵ)	0.2	Rango de recorte en la función objetivo del ratio de política (L^{CLIP}).
Coefficiente de valor (c_1)	0.5	Ponderación del error cuadrático medio de la cabeza de valor (L^{VF}).
Coefficiente de entropía (c_2)	0.01	Ponderación del término de regularización por entropía de la política (L^{ENT}).
Recorte de gradiente (max_grad_norm)	0.5	Umbral máximo para la norma del gradiente global.
Iteraciones de PPO	300	Pasos completos de recolección y actualización del modelo.
Semillas aleatorias (<i>Seed</i>)	10 semillas	Evaluated sobre 10 semillas independientes para validez estadística.
Pool de oponentes (<i>Self-play</i>)	Ratio 80 / 20	80 % de partidas contra el modelo activo, 20 % contra checkpoints pasados.

Cadro 12: Hiperparámetros empleados en el entrenamiento y recolección de trayectorias de PPO.

A.8. Instrucciones de ejecución y reproducibilidad

Esta sección resume el flujo completo para reproducir los experimentos: generación de datos, entrenamiento supervisado, estudio de ablación, PPO y análisis de PPO.

Dependencias y entorno

Todos los comandos ejecutados a continuación se lanzan desde la raíz del repositorio.

Software mínimo: Se requiere Python con entorno virtual activo. El proyecto usa `pyproject.toml` y las dependencias presentes. Se espera también un binario de `stockfish`, presente en `variant-nnue-tools`, para generar/convertir/extraer estadísticas.

Hardware recomendado: GPU NVIDIA con CUDA para entrenamiento (en CPU es funcional pero muy lento). Para batches grandes como los utilizados, (p.ej. `batch_size=16384`) se requiere VRAM alta, un mínimo recomendado de 16GB. En generación de datos, se requiere RAM suficiente para hash de Stockfish (si `-m auto`, el script reserva RAM disponible liberando 2GB).

Supuestos importantes de los scripts

Los scripts de generación de datos, entrenamiento y evaluación asumen ciertas rutas y formatos de dataset. Si se cumplen estas asunciones, la ejecución y reproducción será mucho más rápida:

- `src/dataUtils/run_gen.sh` usa ruta fija a `stockfish`:
`/home/usbt0p/TFG/variant-nnue-tools/src/stockfish`. Se debe ajustar a la ruta apropiada.
- `src/dataUtils/convert.sh` y `src/dataUtils/stats.sh` asumen la presencia de `./variant-nnue-tools/src/stockfish`.
- `create_static_splits.py` asume dataset en bloques de 6 líneas y guarda test en `data/test_splits/`. Esto no debería cambiar, pero si se modifica la estructura de dataset, se debe ajustar el script.
- `train_transformer.py` asume un dataset de texto con formato parseable por `parse_minichess_text_file`, consultar el script para conocer el funcionamiento exacto. Este script crea cachés `.pt` en la misma ruta por defecto o si se le especifica.
- Los modelos utilizados en PPO asumen representación espacial de 28 tokens y usan `pyffish` en las ejecuciones de entorno.

Ejecución recomendada

Asumiendo que se cumplen los requisitos de entorno y dependencias y los supuestos anteriores, la ejecución de scripts y experimentos puede realizarse siguiendo esta guía:

1. Preparar entorno

```
uv sync; source setup.sh
```

2. Generar datos supervisados (.bin)

```
./src/dataUtils/run_gen.sh -v gardner -d 4 -t 16 -m auto
```

3. (Opcional) unir profundidades y deduplicar (si se usan varias profundidades, editar entradas en `merge_datasets_bin.py`)

```
python3 src/dataUtils/merge_datasets_bin.py
```

4. Extraer estadísticas y convertir a texto

```
./src/dataUtils/stats.sh data/gardner_depth4/gen_gardner_d4.bin ↵  
↵ data/gardner_depth4/stats.txt  
./src/dataUtils/convert.sh data/gardner_depth4/gen_gardner_d4.bin ↵  
↵ data/gardner_depth4/gen_gardner_d4.txt
```

5. Crear split estático train/holdout test

```
python3 src/dataUtils/create_static_splits.py ↵  
↵ data/gardner_depth4/gen_gardner_d4.txt
```

Esto genera los archivos: `data/gardner_depth4/d4_val.txt` y `data/test_splits/d4_test.txt`.

6. Entrenamiento supervisado base (Transformer)

```
python3 src/training/train_transformer.py ↵  
↵ data/gardner_depth4/d4_val.txt 64 \  
--num_blocks 3 --epochs 30 --batch_size 512 \  
--attn_backend math --autocast none --lr 0.0030 \  
--representation simple --run_name h2_base --save_dir experiments/h2
```

7. Estudio de ablación supervisado (Hipótesis 2)

```
bash src/experiment_scripts/h2_small.sh  
bash src/experiment_scripts/h2_big.sh  
bash src/experiment_scripts/h2_ablation_seeds.sh
```

(cada script fija representación/cabezas/semillas y guarda logs/checkpoints en `experiments/`).

8. Entrenamiento PPO (Hipótesis 3)

```
python3 src/ppo/run_hypothesis3_experiments.py \  
--variant all \  
--seeds 42,100,2026,999,12345 \  
--iterations 60 \  
--supervised_checkpoint <ruta_al_best_model_supervisado>
```

Este script ejecuta `tabula_rasa` y `pretrained`, y guarda resultados en `results/hypothesis3/`.

9. Análisis y ablaciones de PPO

```
python3 src/evalsAndTournaments/hypothesis3/analyze_hypothesis3.py \  
--results_dir results/hypothesis3  
  
python3 src/evalsAndTournaments/hypothesis3/run_ppo_pool_tournament.py \  
--results_dir results/hypothesis3 --variant both --num_games 100
```